

AI From Zero

Sep 28, 2026 · @Alireza

How to use this book

This book takes you from zero to understanding and building with modern AI, one chapter at a time. It assumes no background in AI, math or programming.

Each chapter has the same four parts:

- **The idea** — a plain-language explanation, usually with an everyday analogy.
- **The picture** — a diagram when the idea has a shape.
- **The code** — short Python you type and run yourself. Reading code teaches little; running and breaking it teaches a lot.
- **Try it** — two or three small exercises. Do them before moving on.

The book has 24 chapters in five parts, plus a math appendix and a concept checklist. Work through them in order; each builds on the last. Ask me whenever you want a chapter expanded, more exercises, or solutions. If a paragraph confuses you, highlight it and leave a comment — I'll rewrite it until it clicks.

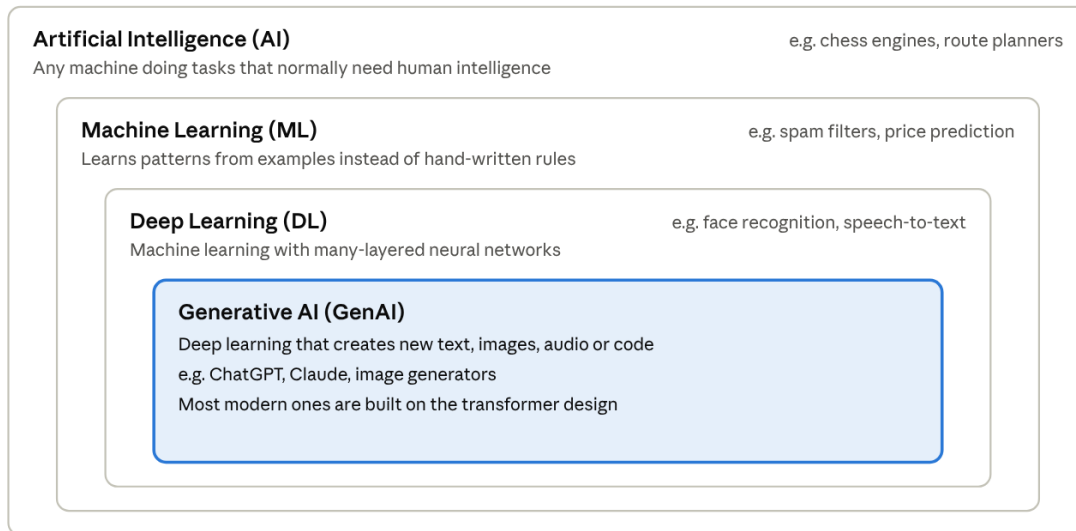
What you need: a computer (or just a browser) and about 30–60 minutes per chapter. Chapter 2 sets everything up.

Chapter 1: The big picture

AI, machine learning, deep learning and generative AI are not four separate things. They are nested layers, each one a more specific kind of the one around it.

1.1 Four words, one family

Generative AI sits inside deep learning, inside machine learning, inside AI



The AI family · four nested layers

Read it from the outside in:

- **Artificial Intelligence (AI)** is the whole field: making computers do things we'd call "smart". Early AI was mostly hand-written rules, like a chess program that follows strategies a human typed in.
- **Machine Learning (ML)** is AI that *learns from examples*. You don't write the rules; you show the computer data and it figures out the rules itself.
- **Deep Learning (DL)** is ML using *neural networks* with many layers. It shines on messy data like images, sound and language.
- **Generative AI (GenAI)** is deep learning that *creates* new things — a paragraph, a picture, a song — instead of only labelling or predicting.

1.2 The big shift: from rules to learning

Imagine building a spam filter. The old way, you write rules: "if the email contains *FREE MONEY*, mark it spam". Spammers change one word and your rule fails.

The machine learning way, you collect 10,000 emails already labelled "spam" or "not spam" and let the computer find the patterns. When spam changes, you retrain on new examples.

	Traditional programming	Machine learning
You provide	Rules + data	Data + correct answers
The computer produces	Answers	The rules (<i>a model</i>)
When the world changes	You rewrite the rules	You retrain on new data

1.3 What is a "model"?

A model is a mathematical function with adjustable knobs, called **parameters**. Data goes in, a prediction comes out, and the knobs decide how one turns into the other.

Training means showing the model examples, checking how wrong its predictions are, and nudging the knobs to be a little less wrong. Repeat millions of times and the model gets good.

A simple model might have 2 knobs. A modern large language model has billions. The idea is the same — you'll build a 2-knob model yourself in Chapter 3.

1.4 What is a transformer? (a first look)

A **transformer** is a particular design of neural network, introduced by Google researchers in 2017 in a paper called *Attention Is All You Need*. Almost every modern language AI — ChatGPT, Claude, Gemini — is built on it. The "T" in GPT stands for Transformer (*Generative Pre-trained Transformer*).

Its core trick is called **attention**. When the model reads a sentence, every word gets to *look at every other word* and decide which ones matter for understanding it. Take this sentence:

The animal didn't cross the street because it was too tired.

What does "it" mean? You know instantly it's the animal, not the street. Attention lets the model learn to connect "it" strongly to "animal". Change "tired" to "wide" and a good model shifts "it" towards "street".

Why was this a breakthrough? Older networks read text one word at a time, left to right, and tended to forget the beginning of long passages. Transformers look at all the words at once. That makes them better at long-range meaning and, crucially, fast to train on modern chips (GPUs). Fast training meant researchers could train them on huge amounts of text — which gave us **large language models (LLMs)**.

An LLM's job sounds almost too simple: **predict the next word**. Given "The cat sat on the", it scores every possible next word and picks a likely one ("mat"). Then it adds that word and predicts again. One word at a time, it writes whole essays. Everything impressive it does comes from getting very, very good at that one task.

(Strictly, models work with **tokens** — words or pieces of words — not whole words. We'll ignore that until Part 4, where we build a transformer piece by piece.)

1.5 A taste of code: the world's tiniest text generator

You don't need anything installed yet. Open [Google Colab](#), click *New notebook*, paste this into a cell, and press the play button.

```
from collections import Counter, defaultdict
import random

text = "the cat sat on the mat the cat ate the fish the dog sat on the rug"
words = text.split()

# Count which word follows which
next_words = defaultdict(Counter)
```

```

for current, following in zip(words, words[1:]):
    next_words[current][following] += 1

print(next_words["the"])  # what tends to come after "the"?

# Generate a sentence, one word at a time
word = "the"
sentence = [word]
for _ in range(8):
    options = next_words[word]
    if not options:
        break
    word = random.choices(list(options), weights=list(options.values()))[0]
    sentence.append(word)

print(" ".join(sentence))

```

This program "learned" from data (it counted word pairs) and it "generates" (it predicts the next word, again and again). That is the same basic loop as an LLM.

The difference is enormous, though. This toy only looks at **one** previous word. A transformer looks at **all** previous words through attention, and learns billions of parameters instead of a small table of counts. Run it a few times — the random choice gives a different sentence each time.

Try it

1. Replace text with a paragraph of your own (a news article works well) and generate again. Does it sound more or less natural?
2. Change the starting word from "the" to another word in your text.
3. Think: why does the output often stop making sense after 4–5 words? (Hint: how much of the sentence does the program remember?)

Key words from this chapter

Term	Plain meaning
Model	A function with adjustable knobs that turns inputs into predictions
Parameters	The knobs; learned during training
Training	Adjusting parameters so predictions become less wrong
Neural network	A model made of layers of simple connected units
Transformer	A neural network design built around attention; the basis of modern LLMs
Attention	The mechanism that lets each word weigh how much every other word matters
LLM	Large language model: a big transformer trained

Term	Plain meaning
	to predict the next token
Token	A word or piece of a word; the unit a language model reads and writes

Chapter 2: Your toolkit

You need exactly one tool to start: Google Colab, a free Python notebook that runs in your browser. Everything in this book runs there.

2.1 Why Python, why notebooks

Python is the language of AI. Nearly every AI library — NumPy, scikit-learn, PyTorch, Hugging Face — is written for it, and its syntax reads almost like English.

A **notebook** mixes code cells and text cells. You run one cell at a time and see the result immediately under it. That makes it perfect for experimenting: change a number, re-run, see what happens.

2.2 Set up in two minutes

1. Go to colab.research.google.com and sign in with a Google account.
2. Click **New notebook**.
3. Type `print("hello, AI")` in the cell and press **Shift + Enter**. The output appears below.
4. Later, for deep learning chapters, you'll switch on a free GPU: *Runtime* → *Change runtime type* → *GPU*.

Prefer working offline? Install Python from python.org and run `pip install jupyter numpy matplotlib scikit-learn`, then start `jupyter notebook`. Colab is simpler, so the book assumes Colab.

2.3 Python in ten minutes

This is only the Python you need for the next few chapters. Run each block in its own cell.

Variables and types — a name that holds a value.

```
hours = 3           # an integer
price = 19.99       # a decimal number (float)
name = "Alireza"    # text (string)
is_ready = True     # True or False (boolean)
print(name, hours * price)
```

Lists — an ordered collection. Data in ML is mostly lists of numbers.

```
scores = [72, 85, 90, 64]
print(scores[0])    # first item: 72
print(len(scores))  # how many: 4
scores.append(99)    # add one
print(sum(scores) / len(scores)) # average
```

Loops — do something for every item. Training a model is one big loop.

```
for s in scores:
    if s >= 80:
        print(s, "pass with distinction")
    else:
        print(s, "pass")
```

Functions — a reusable recipe. A model is, at heart, a function.

```
def predict_price(size_m2):
    return 3000 * size_m2 + 50000

print(predict_price(80))  # 290000
```

Dictionaries — look things up by name.

```
person = {"name": "Sara", "age": 31}
print(person["age"])
```

2.4 NumPy: math on whole lists at once

NumPy is the library that makes AI math fast. Instead of looping over numbers one by one, you do math on entire arrays in one line.

```
import numpy as np

sizes = np.array([50, 80, 120])  # house sizes in m²
prices = 3000 * sizes + 50000    # applies to every item at once
print(prices)                   # [200000 290000 410000]
print(prices.mean())            # average price
```

Notice that `predict_price` from 2.3 and this line do the same thing. The NumPy version works on thousands of houses as fast as on one.

2.5 Matplotlib: seeing your data

In AI you always look at your data before modelling it.

```
import matplotlib.pyplot as plt

plt.scatter(sizes, prices)
plt.xlabel("Size (m²)")
plt.ylabel("Price")
plt.show()
```

Try it

1. Make a list of five temperatures in °C and convert them all to °F with NumPy ($F = C \times 9/5 + 32$).
2. Write a function `is_hot(temp_c)` that returns True above 25.
3. Plot your five temperatures with `plt.plot()` instead of `plt.scatter()`. What's the difference?

Chapter 3: How a machine learns

Every model in this book — from a 2-knob line to a billion-knob transformer — learns with the same loop: guess, measure the error, nudge the knobs, repeat. In this chapter you build that loop from scratch.

3.1 The problem

You have 50 houses with their size and sale price. You want to predict the price of a house you haven't seen. Plot the data and it roughly forms a line: bigger houses cost more.

So our model is a line with two knobs:

$$\text{predicted price} = w \cdot \text{size} + b$$

w (the **weight**) is how much each extra m² adds. b (the **bias**) is the starting price. Training means finding the w and b that fit the data best.

3.2 Measuring "wrong": the loss

For each house, compare the prediction with the real price. Square the difference (so negatives don't cancel positives, and big misses count extra), then average over all houses. This number is the **loss** — here, the *mean squared error*:

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$\hat{y}$ is the prediction, y is the real price, n is the number of houses. Lower loss = better model. Training is simply "make L as small as possible".

3.3 Walking downhill: gradient descent

Imagine standing on a foggy hillside, trying to reach the valley. You can't see far, but you can feel which way the ground slopes under your feet. So you take a small step downhill, feel again, step again.

That's **gradient descent**. The "slope" is the **gradient**: it tells us, for each knob, whether turning it up would raise or lower the loss. We turn each knob a little in the direction that lowers it:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w} \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

η (eta) is the **learning rate**: the size of each step. Too small and training crawls. Too big and you overshoot the valley and bounce around.

You don't need to calculate gradients by hand — libraries do it automatically. For this simple model the formulas are short, so we write them out once to see there's no magic.

3.4 The code: a model that learns

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Make some data: 50 houses, true rule = 3000 per m² + 50,000, plus noise
np.random.seed(0)
sizes = np.random.uniform(40, 150, 50)
prices = 3000 * sizes + 50000 + np.random.normal(0, 20000, 50)

# Scale numbers down so training is stable
x = sizes / 100
```

```

y = prices / 100000

# 2. Start with knobs at zero
w, b = 0.0, 0.0
learning_rate = 0.1

# 3. The training loop
for step in range(1000):
    pred = w * x + b                # guess
    error = pred - y
    loss = (error ** 2).mean()      # measure
    grad_w = 2 * (error * x).mean() # which way is downhill?
    grad_b = 2 * error.mean()
    w -= learning_rate * grad_w     # nudge the knobs
    b -= learning_rate * grad_b
    if step % 200 == 0:
        print(f"step {step}: loss {loss:.4f}")

print(f"Learned: price ≈ {w * 1000:.0f} × size + {b * 100000:.0f}")

# 4. See the result
plt.scatter(sizes, prices, label="real houses")
plt.plot(sizes, (w * x + b) * 100000, color="red", label="model")
plt.xlabel("Size (m²)"); plt.ylabel("Price"); plt.legend(); plt.show()

```

You should see the loss drop fast (about 12.7 to 0.03) and the model learn roughly **2,944 per m² + 52,000**. That's close to the hidden true rule of 3,000 + 50,000; the gap comes from the random noise in the data.

Nobody told the program the rule. It found it from examples. That is machine learning.

3.5 The same thing in three lines

In practice you use a library. **scikit-learn** is the standard one for classic ML:

```

from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(sizes.reshape(-1, 1), prices)
print(model.coef_[0], model.intercept_)    # about 2944 and 52090
print(model.predict([[100]]))              # price of a 100 m² house

```

Same answer. Now you know what's happening inside.

3.6 How this connects to transformers

A transformer like the ones behind ChatGPT or Claude trains with *exactly this loop*. The differences are scale, not kind:

	Our house model	A large language model
Knobs (parameters)	2	Billions
Input	One number (size)	A long sequence of tokens
Prediction	A price	The next token

	Our house model	A large language model
Loss	Mean squared error	Cross-entropy (how surprised it was by the real next token)
Learning method	Gradient descent	Gradient descent (a smarter variant called Adam)

Try it

1. Change `learning_rate` to `0.01`, then to `1.5`. What happens to the loss in each case? Why?
2. Change the true rule in step 1 (say 5000 per m²) and check the model finds it.
3. Increase the noise from `20000` to `80000`. How close does the model get now?

Part 2 — Classic machine learning

Chapter 4: Data and features

A model can only be as good as its data. Practitioners often say 80% of ML work is preparing data, and only 20% is modelling.

4.1 The vocabulary of a dataset

A dataset is a table. Each **row** is one example (a passenger, an email, a house). Each **column** is a **feature** — a measurable property. The column you want to predict is the **label** or **target**.

Term	Meaning	Titanic example
Example / sample	One row	One passenger
Feature (X)	An input column	Age, ticket class, fare
Label / target (y)	What we predict	Survived (1) or not (0)
Numerical feature	A number with meaning in its size	Age, fare
Categorical feature	One of a fixed set of values	Sex, port of embarkation

Data also comes as text, images, audio and time series. Classic ML mostly works on tables; deep learning (Part 3) handles the rest.

4.2 Load and explore (EDA)

Exploratory data analysis (EDA) means looking at your data before modelling. We'll use the Titanic passenger list and the **pandas** library, Python's spreadsheet.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset("titanic")
```

```
print(df.shape)      # (891, 15): 891 passengers, 15 columns
print(df.head())     # first five rows
df.info()            # column types and non-missing counts
print(df.describe()) # min, max, mean of numeric columns
print(df.isna().sum()) # missing values per column
print(df["survived"].value_counts(normalize=True)) # 62% died, 38% survived
```

Things to look for: missing values, impossible values (a negative age), extreme **outliers**, and how balanced the label is.

4.3 Clean: missing values and outliers

About 20% of ages are missing. You have three options: drop those rows, fill them with a typical value (**imputation**), or predict them with another model. Filling with the median is a simple, robust start.

```
df = df[["survived", "pclass", "sex", "age", "sibsp", "fare", "embarked"]].copy()
df["age"] = df["age"].fillna(df["age"].median())
df = df.dropna(subset=["embarked"]) # only 2 rows missing: just drop them
```

Outliers (a fare of 512 when most are under 50) aren't always errors. Check whether they're real before removing them.

4.4 Feature engineering

Feature engineering means creating new, more useful columns from existing ones. Good features often beat fancier models.

```
df["family_size"] = df["sibsp"] + 1 # siblings/spouses aboard + the passenger
```

Other common tricks: extracting the day of week from a date, the length of a text, or a ratio of two columns.

4.5 Encode categories as numbers

Models only understand numbers. **One-hot encoding** turns a category into 0/1 columns, one per value:

```
df = pd.get_dummies(df, columns=["sex", "embarked"], drop_first=True)
# "sex" becomes sex_male (1 or 0); "embarked" becomes embarked_Q, embarked_S
```

For ordered categories (small < medium < large) use **ordinal encoding**: 0, 1, 2. Don't do this for unordered ones — the model would think "Southampton > Cherbourg".

4.6 Split: train, validation, test

This is the most important habit in ML. You must test your model on data it has **never seen**, or you'll fool yourself.

- **Training set** (~70–80%): the model learns from it.
- **Validation set** (~10–15%): you use it to compare models and choose settings.
- **Test set** (~10–20%): touched **once**, at the very end, for an honest final score.

```
from sklearn.model_selection import train_test_split

X = df.drop(columns="survived")
y = df["survived"]
X_train, X_test, y_train, y_test = train_test_split(
```

```
X, y, test_size=0.2, random_state=42, stratify=y)
print(X_train.shape, X_test.shape)  # (711, 8) (178, 8)
```

stratify=y keeps the same survival ratio in both sets. random_state makes the split repeatable.
(Chapter 8 replaces the single validation set with cross-validation.)

4.7 Scale numeric features

Age runs 0–80 while fare runs 0–512. Many models (kNN, SVM, neural networks, anything using gradient descent) work badly when features have very different ranges.

- **Standardization:** shift and scale to mean 0, standard deviation 1. The usual default.
- **Normalization (min-max):** squeeze into 0–1. Common for images and neural nets.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X_train)      # learn mean/std from TRAIN only
X_train_s = scaler.transform(X_train)
X_test_s = scaler.transform(X_test)         # apply the same numbers to test
```

4.8 The silent killer: data leakage

Data leakage is when information from the test set, or from the future, sneaks into training. Your score looks great; the real-world model fails.

Common causes: fitting the scaler on all data before splitting, filling missing values with a mean computed over all rows, or including a feature that wouldn't exist at prediction time ("was refunded" when predicting fraud). The rule: **split first, then fit every preprocessing step on the training set only**. Chapter 8's pipelines enforce this automatically.

Try it

1. Instead of the median, fill missing ages with the median age *per ticket class* (`df.groupby("pclass")["age"].transform("median")`).
2. Create a feature `is_alone` that is 1 when `family_size == 1`.
3. Find a leakage risk: if you were predicting hospital readmission, which of these features would leak — age, diagnosis, "number of days readmitted"?

Chapter 5: Supervised learning

Supervised learning means learning from labelled examples: every training row comes with the right answer. It covers two task types — **regression** (predict a number) and **classification** (predict a category).

5.1 The four ways machines learn

Type	Learns from	Example
Supervised	Inputs + correct answers	Spam filter, price prediction
Unsupervised	Inputs only; finds structure	Customer segments (Chapter 7)
Self-supervised	Inputs that label themselves	An LLM predicting the next

Type	Learns from	Example
		word of real text (Part 4)
Reinforcement	Rewards from trial and error	Game-playing agents (Chapter 20)

5.2 Regression: predicting numbers

You met **linear regression** in Chapter 3. With many features it becomes one weight per feature:

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Ridge and **Lasso** are linear regression plus a penalty that keeps weights small, which fights overfitting (Chapter 8). Lasso can push useless weights all the way to zero.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_absolute_error

X, y = load_diabetes(return_X_y=True)  # predict disease progression from 10
measurements
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

for model in [LinearRegression(), Ridge(alpha=1.0), Lasso(alpha=0.1)]:
    model.fit(X_tr, y_tr)
    print(type(model).__name__, mean_absolute_error(y_te, model.predict(X_te)))
```

5.3 Classification: predicting categories

Logistic regression, despite its name, is a classifier. It computes a linear score, then squashes it into a probability between 0 and 1 with the **sigmoid** function:

$$P(y = 1) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = w \cdot x + b$$

If the probability is above 0.5 (the **decision threshold**), predict class 1. Its loss is **cross-entropy** (log loss): it punishes being confidently wrong very hard. The same loss trains LLMs.

For more than two classes, **softmax** generalizes sigmoid: it turns a score per class into probabilities that sum to 1.

5.4 Four classic classifiers

- **k-nearest neighbours (kNN)** — no real training; to classify a new point, look at the k most similar training examples and take a vote. Simple, but slow on big data and needs scaled features.
- **Support vector machine (SVM)** — finds the boundary with the widest possible gap (the *margin*) between classes. With the **kernel trick** it can draw curved boundaries.
- **Naïve Bayes** — uses probability (Bayes' theorem) and assumes features are independent. Fast, a classic for text classification.
- **Logistic regression** — the reliable, interpretable baseline. Always try it first.

5.5 Compare them on the Titanic

Use `X_train_s`, `X_test_s`, `y_train`, `y_test` from Chapter 4.

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB

models = {
    "Logistic regression": LogisticRegression(),
    "k-nearest neighbours": KNeighborsClassifier(n_neighbors=5),
    "SVM": SVC(),
    "Naive Bayes": GaussianNB(),
}
for name, model in models.items():
    model.fit(X_train_s, y_train)
    print(f"{name:22s} accuracy: {model.score(X_test_s, y_test):.3f}")
```

Expect roughly 0.81, 0.80, 0.83 and 0.73. Notice every scikit-learn model has the same interface: `fit()` to train, `predict()` to use, `score()` to evaluate. Learn it once, use it everywhere.

5.6 Which model when?

Situation	Good first choice
Need to explain every decision	Logistic / linear regression
Tabular data, want best accuracy	Gradient boosting (Chapter 6)
Short texts, little data	Naive Bayes, logistic regression
Small dataset, complex boundary	SVM
Images, audio, long text	Deep learning (Part 3)

Try it

1. Change `n_neighbors` in kNN to 1, 15 and 50. Which is best? Why is 1 risky?
2. Train kNN on *unscaled* `X_train` and compare. How much does scaling matter?
3. Print `LogisticRegression().fit(X_train_s, y_train).coef_`. Which feature pushes survival up most?

Chapter 6: Trees and ensembles

For tabular data (spreadsheets, databases), tree ensembles — random forests and gradient boosting — are usually the most accurate models, often beating deep learning.

6.1 Decision trees: twenty questions

A **decision tree** predicts by asking a series of yes/no questions: "Is the passenger male? Is their age under 4? Are they in 3rd class?" Each question splits the data; each final **leaf** gives a prediction.

Training picks, at each step, the question that best separates the classes, measured by **Gini impurity** or **entropy** (how mixed the classes are in each group).

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
tree = DecisionTreeClassifier(max_depth=3, random_state=42)
tree.fit(X_train, y_train)      # trees don't need scaled features
print(tree.score(X_test, y_test))  # about 0.81
print(export_text(tree, feature_names=list(X_train.columns)))
```

The printout shows the learned questions. The first split is `sex_male` — the tree rediscovered "women and children first". Trees are very **interpretable**, but a deep tree memorizes the training data (overfits). `max_depth` limits that.

6.2 Ensembles: the wisdom of crowds

An **ensemble** combines many models so their errors cancel out. Two main strategies:

- **Bagging** (bootstrap aggregating): train many models in parallel, each on a random sample of the data, then average their votes. Reduces variance.
- **Boosting**: train models one after another, each focusing on the mistakes of the ones before. Reduces bias.

6.3 Random forests

A **random forest** is bagging with decision trees, plus a twist: each split may only consider a random subset of features. That makes the trees different from each other, and the average far more robust than any one tree.

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd
```

```
forest = RandomForestClassifier(n_estimators=300, random_state=42)
forest.fit(X_train, y_train)
print(forest.score(X_test, y_test))  # about 0.82
```

```
importance = pd.Series(forest.feature_importances_, index=X_train.columns)
print(importance.sort_values(ascending=False))  # fare, age, sex at the top
```

Feature importance tells you which inputs the model relies on most — useful for explaining results and for spotting leakage (a suspiciously dominant feature).

6.4 Gradient boosting

Gradient boosting builds shallow trees one at a time. Each new tree predicts the *remaining errors* of the ensemble so far, and is added with a small weight (the learning rate). It's gradient descent (Chapter 3) done with trees.

```
from sklearn.ensemble import HistGradientBoostingClassifier
```

```
gb = HistGradientBoostingClassifier(random_state=42)
gb.fit(X_train, y_train)
print(gb.score(X_test, y_test))  # about 0.83
```

In industry and Kaggle competitions, the go-to libraries are **XGBoost**, **LightGBM** and **CatBoost** (`pip install xgboost`). They're faster and have more options, but work the same way.

6.5 Key settings (hyperparameters)

Setting	Effect of increasing it
<code>n_estimators</code> (number of trees)	More stable; slower. For boosting, too many can overfit
<code>max_depth</code>	Trees can learn more complex patterns; more overfitting risk
<code>min_samples_leaf</code>	Each leaf must cover more examples; smoother, less overfitting
<code>learning_rate</code> (boosting)	Faster learning; less precise. Small rate + more trees usually wins

Chapter 8 shows how to choose these automatically.

Try it

1. Train a tree with no `max_depth`. Compare its training score (`tree.score(X_train, y_train)`) with its test score. What do you see?
2. Plot the tree visually with `from sklearn.tree import plot_tree; plot_tree(tree, feature_names=list(X_train.columns), filled=True)`.
3. Install XGBoost and train `XGBClassifier()` on the same data.

Chapter 7: Unsupervised learning

Unsupervised learning finds structure in data that has no labels. The two main jobs are **clustering** (grouping similar things) and **dimensionality reduction** (compressing many features into a few).

7.1 Clustering with k-means

k-means splits data into k groups. It starts with k random centre points, assigns every point to its nearest centre, moves each centre to the middle of its points, and repeats until nothing moves.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=300, centers=4, random_state=42) # we pretend we don't
know there are 4
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X)

plt.scatter(X[:, 0], X[:, 1], c=km.labels_)
plt.scatter(*km.cluster_centers_.T, marker="x", s=200, color="black")
plt.show()
```

Real uses: customer segmentation, grouping similar documents, compressing image colours.

7.2 Choosing k

You usually don't know how many groups exist. Two tools help:

- **Elbow method** — plot `km.inertia_` (total distance from points to their centre) for $k = 1, 2, 3, \dots$. It always falls; pick the k where it stops falling steeply.
- **Silhouette score** — from -1 to 1 : how much closer each point is to its own cluster than to the next one. Higher is better.

```
from sklearn.metrics import silhouette_score

for k in range(2, 7):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    print(k, round(km.inertia_), round(silhouette_score(X, km.labels_), 3))
# k=4 gives the best silhouette (about 0.79)
```

7.3 Other clustering methods

Method	Idea	Good when
k-means	Nearest centre	Round, similar-sized groups
Hierarchical	Merge closest groups step by step into a tree (dendrogram)	You want to see groups at every level
DBSCAN	Dense regions are clusters; sparse points are noise	Odd shapes, outliers, unknown k
Gaussian mixture	Each cluster is a bell curve; soft membership	Overlapping groups

7.4 Dimensionality reduction with PCA

A handwritten-digit image of 8×8 pixels has 64 features. Humans can't picture 64 dimensions. **Principal Component Analysis (PCA)** finds the few directions along which the data varies most, and projects onto them.

```
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

digits = load_digits()                # 1,797 images, 64 features each
pca = PCA(n_components=2)
X2 = pca.fit_transform(digits.data)
print(pca.explained_variance_ratio_.sum())  # about 0.29 of the information kept in
just 2 numbers

plt.scatter(X2[:, 0], X2[:, 1], c=digits.target, cmap="tab10", s=8)
plt.colorbar(); plt.show()
```

Even in 2D, the digits form visible groups. Uses of PCA: visualization, removing noise, and speeding up other models.

For visualization only, **t-SNE** and **UMAP** usually give clearer pictures than PCA because they preserve neighbourhoods rather than straight-line directions.

7.5 Anomaly detection

A close cousin: finding rows that don't fit any pattern — fraud, faulty sensors, network attacks. `sklearn.ensemble.IsolationForest` is the standard starting point.

Try it

1. Run k-means on the digits with `k = 10` and compare `km.labels_` with `digits.target`. Did it find the digits without being told?
2. Try `PCA(n_components=0.95)`: how many components keep 95% of the variance?
3. Replace PCA with `from sklearn.manifold import TSNE; TSNE(n_components=2).fit_transform(digits.data)` and compare the plot.

Chapter 8: Training classical models well

Training a model is one line (`model.fit`). Training it *well* — so it works on data it has never seen — is the real skill. This chapter is the full workflow.

8.1 The training workflow, end to end

1. Define the problem and the metric that matters (Chapter 9).
2. Collect data, then split off a test set and lock it away.
3. Explore and clean the training data (Chapter 4).
4. Build a **baseline**: the simplest reasonable model (or even "always predict the most common class").
5. Try a few model families with cross-validation.
6. Tune the best one's hyperparameters.
7. Evaluate once on the test set.
8. Retrain on all the data, save it, deploy it (Part 5), monitor it.

8.2 Overfitting and underfitting

- **Underfitting**: the model is too simple to capture the pattern. Bad on training data, bad on test data.
- **Overfitting**: the model memorizes the training data, including its noise. Great on training data, bad on test data.
- The goal is the sweet spot between: it **generalizes**.

This is the **bias–variance trade-off**. *Bias* is error from wrong assumptions (too simple). *Variance* is error from sensitivity to the particular training sample (too complex).

See it happen by fitting curves of different complexity to 30 noisy points:

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
```

```
x = np.sort(rng.uniform(0, 1, 30)).reshape(-1, 1)
y = np.sin(2 * np.pi * x).ravel() + rng.normal(0, 0.2, 30)
x_tr, x_te, y_tr, y_te = train_test_split(x, y, test_size=0.3, random_state=0)

for degree in [1, 4, 15]:
    m = make_pipeline(PolynomialFeatures(degree), LinearRegression()).fit(x_tr, y_tr)
    print(degree, "train:", round(mean_squared_error(y_tr, m.predict(x_tr)), 3),
          "test:", round(mean_squared_error(y_te, m.predict(x_te)), 3))
```

Degree 1 (a straight line) underfits. Degree 4 fits well on both. Degree 15 gets almost zero training error but a test error hundreds of times worse — textbook overfitting.

Symptom	Diagnosis	Fixes
Train error high, test error high	Underfitting	More complex model, better features, less regularization
Train error low, test error high	Overfitting	More data, simpler model, regularization, early stopping
Both low, close together	Good fit	Ship it (after the test set confirms)

8.3 Regularization

Regularization adds a penalty for complexity to the loss, so the model prefers simpler explanations.

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_j w_j^2 \quad (\text{L2 / Ridge}) \quad L_{\text{total}} = L_{\text{data}} + \lambda \sum_j |w_j| \quad (\text{L1 / Lasso})$$

λ controls the strength (alpha in scikit-learn, and $C = 1/\lambda$ in logistic regression and SVM). For trees, regularization means limits like `max_depth` and `min_samples_leaf`. In neural networks you'll meet **dropout** and **weight decay** (Chapter 11).

8.4 Cross-validation

A single validation split is noisy: you might just get lucky. **k-fold cross-validation** splits the training data into k parts, trains k times (each time holding out a different part), and averages the scores.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(pipe, X_train, y_train, cv=5) # pipe is built in 8.5
print(scores.round(3), "mean:", scores.mean().round(3), "±", scores.std().round(3))
# e.g. [0.79 0.72 0.824 0.838 0.803] mean: 0.795 ± 0.041
```

The spread ($\pm$) tells you how much to trust the difference between two models. Variants: **stratified k-fold** for classification (the default), **group k-fold** when rows from one customer must stay together, and **time-series split** when the future must never train the past.

8.5 Pipelines: preprocessing without leakage

A **pipeline** chains preprocessing and the model into one object. Every step is fitted on the training folds only, so cross-validation can't leak. This version starts from the *raw* Titanic data:

```

import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

raw = sns.load_dataset("titanic")[["survived", "pclass", "sex", "age", "sibsp",
"fare", "embarked"]]
X, y = raw.drop(columns="survived"), raw["survived"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42, stratify=y)

numeric = ["pclass", "age", "sibsp", "fare"]
categorical = ["sex", "embarked"]
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
numeric),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
OneHotEncoder(handle_unknown="ignore")), categorical),
])
pipe = Pipeline([("prep", prep), ("model", RandomForestClassifier(random_state=42))])

```

8.6 Hyperparameter tuning

Parameters are learned by training (weights, tree splits). **Hyperparameters** are settings you choose before training (tree depth, learning rate, k). Tuning means searching for the best hyperparameters using cross-validation.

```

from sklearn.model_selection import GridSearchCV

grid = GridSearchCV(pipe, {
    "model__n_estimators": [100, 300],
    "model__max_depth": [3, 5, None],
    "model__min_samples_leaf": [1, 5],
}, cv=5, n_jobs=-1)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)    # best CV score about 0.82
print(grid.score(X_test, y_test))             # final, honest test score about 0.80

```

- **Grid search** tries every combination — thorough, slow.
- **Random search** (RandomizedSearchCV) samples combinations — usually as good, much faster.
- **Bayesian optimization** (the **Optuna** library) learns which regions look promising — best for expensive models.

Notice the test score is slightly below the CV score. That's normal: tuning picks the setting that happened to do best on those folds, which flatters it a little.

8.7 Other training issues

- **Class imbalance** (e.g. 1% fraud): use `class_weight="balanced"`, resample the data (oversample the rare class or undersample the common one), and judge with precision/recall, not accuracy.

- **Learning curves:** plot score vs training-set size (`sklearn.model_selection.learning_curve`). If test score still rises at the end, more data will help.
- **Reproducibility:** fix `random_state`, record library versions and the exact data used.
- **Saving a model:** `import joblib; joblib.dump(grid.best_estimator_, "model.joblib")`, then `joblib.load("model.joblib")` to use it later. Chapter 22 serves this file from a Docker container.

Try it

1. Replace `GridSearchCV` with `RandomizedSearchCV(pipe, ..., n_iter=10)`. Is it as good? How much faster?
2. Swap the model in the pipeline for `LogisticRegression(max_iter=1000)` and tune `model__C` over `[0.01, 0.1, 1, 10]`.
3. Plot a learning curve for the pipeline. Would more passengers help?

Chapter 9: Evaluating classical models

A single accuracy number can hide a useless model. Evaluation means choosing the metric that matches the real cost of mistakes, comparing against a baseline, and testing on truly unseen data.

9.1 Always start with a baseline

```
from sklearn.dummy import DummyClassifier

dummy = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)
print(dummy.score(X_test, y_test))  # about 0.62: "everyone dies" is 62% accurate!
```

Our tuned forest scores about 0.80. The honest improvement is $0.62 \rightarrow 0.80$, not $0 \rightarrow 0.80$. If 99% of transactions are legitimate, a model that never flags fraud is 99% accurate and worthless.

9.2 The confusion matrix

For a yes/no classifier, every prediction is one of four outcomes:

	Predicted: no	Predicted: yes
Actually no	True negative (TN)	False positive (FP) — a false alarm
Actually yes	False negative (FN) — a miss	True positive (TP)

```
from sklearn.metrics import confusion_matrix, classification_report

best = grid.best_estimator_  # from Chapter 8
pred = best.predict(X_test)
print(confusion_matrix(y_test, pred))  # [[104  6]
                                       # [ 29 40]]
print(classification_report(y_test, pred, digits=3))
```

9.3 Classification metrics

Metric	Question it answers	Formula
--------	---------------------	---------

Metric	Question it answers	Formula
Accuracy	What fraction of all predictions were right?	$(TP + TN) / \text{all}$
Precision	When it says "yes", how often is it right?	$TP / (TP + FP)$
Recall (sensitivity)	Of the real "yes" cases, how many did it catch?	$TP / (TP + FN)$
Specificity	Of the real "no" cases, how many did it clear?	$TN / (TN + FP)$
F1 score	A single balance of precision and recall	$2 \times P \times R / (P + R)$
ROC-AUC	How well does it rank positives above negatives, across all thresholds?	Area under the ROC curve; 0.5 = random, 1 = perfect
PR-AUC	Same idea, focused on the rare positive class	Area under the precision–recall curve
Log loss	Are its probabilities trustworthy?	Cross-entropy; lower is better

Our model: precision 0.87 but recall only 0.58 for survivors — when it predicts survival it's usually right, but it misses 42% of actual survivors.

Which to prioritize depends on the cost of errors:

- **Recall** when misses are expensive: cancer screening, fraud, a driver falling asleep.
- **Precision** when false alarms are expensive: spam filters (losing a real email), recommending a costly intervention.
- **F1** or **PR-AUC** when classes are imbalanced and both matter.
- For **multi-class** problems, report per-class scores and a *macro* average (every class equal) or *weighted* average (by class size).

9.4 The threshold is a choice

The classifier outputs a probability; 0.5 is just the default cut-off. Moving it trades precision against recall:

```
from sklearn.metrics import precision_score, recall_score, roc_auc_score

proba = best.predict_proba(X_test)[: , 1]
print("ROC-AUC:", roc_auc_score(y_test, proba))    # about 0.84
for t in [0.3, 0.5, 0.7]:
    p = (proba >= t).astype(int)
    print(t, "precision", round(precision_score(y_test, p), 2), "recall",
          round(recall_score(y_test, p), 2))
```

```
# 0.3 -> precision 0.66, recall 0.81
# 0.5 -> precision 0.87, recall 0.58
```

Pick the threshold on validation data according to the business cost, never on the test set.

9.5 Regression metrics

Metric	Meaning	Note
MAE (mean absolute error)	Average size of the error, in the target's units	Easy to explain: "off by \$12,000 on average"
RMSE (root mean squared error)	Like MAE but punishes big errors more	Use when large misses are especially bad
MAPE	Average error as a percentage	Breaks when true values are near zero
R ²	Fraction of variance explained; 1 = perfect, 0 = no better than predicting the mean	Can be negative for a terrible model

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
# mae = mean_absolute_error(y_true, y_pred); rmse = mean_squared_error(y_true,
y_pred) ** 0.5; r2 = r2_score(y_true, y_pred)
```

9.6 Evaluating clustering

Without labels, use internal scores: **silhouette** (higher is better), **Davies–Bouldin** (lower is better). With labels, **adjusted Rand index** compares clusters with the truth. Most importantly: do the clusters make sense to a domain expert?

9.7 Beyond the metric

- **Calibration**: when the model says 70%, does it happen 70% of the time? Check with `sklearn.calibration.CalibrationDisplay`.
- **Slice analysis**: compute metrics per group (age band, region, device). A good average can hide a group where the model fails (Chapter 24).
- **Error analysis**: read 20–50 wrong predictions by hand. It's the fastest way to find bad labels, leakage or missing features.
- **Statistical confidence**: small test sets give noisy scores. A 1-point difference on 178 passengers is probably noise; use cross-validation spreads or bootstrap confidence intervals.
- **Online evaluation**: after deployment, measure the real outcome with **A/B tests** and monitor for **drift** (Chapter 23).

Try it

1. Plot the ROC curve with `from sklearn.metrics import RocCurveDisplay; RocCurveDisplay.from_predictions(y_test, proba)`.
2. Find the threshold that gives recall of at least 0.75. What precision do you get?
3. Compute the forest's accuracy separately for men and women. Is it equally good for both?

Part 3 — Deep learning

Chapter 10: Neural networks and backpropagation

A neural network is many simple units stacked in layers. Each unit does logistic regression's job — a weighted sum and a squashing function — and stacking them lets the network learn patterns no single line can.

10.1 The artificial neuron

A **neuron** takes inputs, multiplies each by a weight, adds a bias, and passes the result through an **activation function**:

$$\text{output} = f(w_1x_1 + w_2x_2 + \dots + w_nx_n + b)$$

The activation adds **non-linearity**. Without it, a stack of layers collapses into one big linear equation and can only draw straight lines.

Activation	Formula (plain)	Where it's used
ReLU	$\max(0, z)$	Default for hidden layers: simple and fast
Sigmoid	$1 / (1 + e^{-z})$, output 0–1	Output for yes/no problems
Tanh	Like sigmoid but –1 to 1	Older networks, RNNs
Softmax	Turns scores into probabilities summing to 1	Output for multi-class problems (and LLMs)
GELU	A smooth ReLU	Transformers

10.2 Layers

A network has an **input layer** (your features), one or more **hidden layers**, and an **output layer** (the prediction). "Deep" learning simply means many hidden layers. Each layer transforms the data into a more useful representation: in an image network, early layers detect edges, middle layers shapes, late layers whole objects.

Passing an input through all layers to get a prediction is the **forward pass**.

10.3 Backpropagation

Training works like Chapter 3: compute the loss, find the gradient, step downhill. The problem is that a deep network has millions of weights spread across layers. How much did a weight in layer 1 contribute to the final error?

Backpropagation answers this with the **chain rule** from calculus. It starts at the loss and works backwards layer by layer, passing along "how much the loss changes if this value changes". One

backward pass gives the gradient for every weight at once. It's the engine behind all deep learning, including LLMs.

10.4 Build one from scratch: solving XOR

XOR outputs 1 when exactly one input is 1. No straight line separates the classes, so logistic regression fails. A network with one hidden layer solves it.

```
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([[0], [1], [1], [0]], dtype=float)

rng = np.random.default_rng(42)
W1 = rng.normal(0, 1, (2, 4)); b1 = np.zeros((1, 4))    # 2 inputs -> 4 hidden neurons
W2 = rng.normal(0, 1, (4, 1)); b2 = np.zeros((1, 1))    # 4 hidden -> 1 output
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 1.0

for step in range(5000):
    # forward pass
    h = np.tanh(X @ W1 + b1)
    p = sigmoid(h @ W2 + b2)
    loss = -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))    # cross-entropy

    # backward pass (chain rule, layer by layer)
    dz2 = (p - y) / len(X)
    dW2 = h.T @ dz2; db2 = dz2.sum(0, keepdims=True)
    dz1 = (dz2 @ W2.T) * (1 - h ** 2)    # derivative of tanh
    dW1 = X.T @ dz1; db1 = dz1.sum(0, keepdims=True)

    # gradient descent step
    W1 -= lr * dW1; b1 -= lr * db1
    W2 -= lr * dW2; b2 -= lr * db2
    if step % 1000 == 0:
        print(step, round(loss, 4))

print(p.round(3).ravel())    # [0. 0.999 1. 0.001] -> XOR learned
```

That's a complete neural network: forward pass, loss, backward pass, update. Every framework automates exactly these steps.

10.5 Key training concepts

- **Epoch:** one full pass over the training data.
- **Batch:** a small group of examples processed together (e.g. 32). **Mini-batch gradient descent** updates the weights after each batch — faster and less memory than using all data at once.
- **Weight initialization:** weights start small and random. All zeros would make every neuron learn the same thing.
- **Vanishing / exploding gradients:** in very deep networks, gradients can shrink to nothing or blow up as they pass back through many layers. ReLU, careful initialization, normalization layers and **residual connections** (shortcuts that skip layers) fix this — all used in transformers.

- **Universal approximation:** a big enough network can approximate almost any function. The hard part is training it to generalize.

Try it

1. Change the hidden layer from 4 neurons to 2. Does it still learn XOR? Try a few random seeds.
2. Set `lr = 0.01`. How many more steps does it need?
3. Replace `np.tanh` with `ReLU` (`np.maximum(0, z)`). What must change in the backward pass? (Hint: `ReLU`'s derivative is 1 where $z > 0$, else 0.)

Chapter 11: Training deep networks in PyTorch

PyTorch automates what you did by hand in Chapter 10: it computes gradients for you (**autograd**), runs on GPUs, and provides ready-made layers. It's the framework behind most modern AI research, including most LLMs.

In Colab, switch on a GPU first: *Runtime* → *Change runtime type* → *GPU*. PyTorch is pre-installed.

11.1 Tensors and autograd

A **tensor** is PyTorch's array — like a NumPy array, but it can live on a GPU and remember how it was computed.

```
import torch

w = torch.tensor(3.0, requires_grad=True)    # track gradients for w
x = torch.tensor(2.0)
loss = (w * x - 10) ** 2                    # prediction 6, target 10
loss.backward()                             # backpropagation, automatically
print(w.grad)                              # tensor(-16.): increase w to reduce loss
```

11.2 A complete training script

We'll classify **Fashion-MNIST**: 70,000 small greyscale images of 10 clothing types. For simplicity we use the test set for validation here; in real work, split off a separate validation set.

```
import torch, torch.nn as nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

device = "cuda" if torch.cuda.is_available() else "cpu"

# 1. Data: Dataset + DataLoader (batches and shuffling)
tfm = transforms.ToTensor()
train_ds = datasets.FashionMNIST("data", train=True, download=True, transform=tfm)
test_ds = datasets.FashionMNIST("data", train=False, download=True, transform=tfm)
train_dl = DataLoader(train_ds, batch_size=64, shuffle=True)
test_dl = DataLoader(test_ds, batch_size=256)

# 2. Model
model = nn.Sequential(
    nn.Flatten(),                                # 28x28 image -> 784 numbers
    nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.2),
```

```

    nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(128, 10),                                # one score per class
).to(device)

# 3. Loss and optimizer
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)

def evaluate(loader):
    model.eval()                                # turns dropout off
    correct, total_loss = 0, 0.0
    with torch.no_grad():                        # no gradients needed
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            out = model(xb)
            total_loss += loss_fn(out, yb).item() * len(xb)
            correct += (out.argmax(1) == yb).sum().item()
    n = len(loader.dataset)
    return total_loss / n, correct / n

# 4. The training loop
for epoch in range(5):
    model.train()
    for xb, yb in train_dl:
        xb, yb = xb.to(device), yb.to(device)
        loss = loss_fn(model(xb), yb)           # forward pass + loss
        optimizer.zero_grad()                   # clear old gradients
        loss.backward()                          # backward pass
        optimizer.step()                         # update weights
    val_loss, val_acc = evaluate(test_dl)
    print(f"epoch {epoch + 1}: val loss {val_loss:.3f}, val accuracy {val_acc:.3f}")

torch.save(model.state_dict(), "fashion_mlp.pt") # save the trained weights

```

Expect roughly 87–89% accuracy after 5 epochs. The five lines inside the loop — forward, loss, zero_grad, backward, step — are the same whether you train this tiny model or a 70-billion-parameter LLM.

11.3 Optimizers

Optimizer	Idea	When
SGD	Plain gradient descent on mini-batches	Simple; still used for some vision models
SGD + momentum	Keeps a running direction, like a rolling ball	Smoother, faster than plain SGD
Adam	Momentum + a separate adaptive step size per weight	The safe default
AdamW	Adam with weight decay done correctly	Standard for transformers and LLMs

11.4 The learning rate: the most important setting

If you tune only one thing, tune the learning rate. Common tools:

- **Learning-rate schedules:** start higher, decay over time (`torch.optim.lr_scheduler.CosineAnnealingLR`).
- **Warmup:** start tiny and ramp up over the first few hundred steps. Essential for transformers.
- **Gradient clipping:** cap gradient size to avoid sudden blow-ups (`torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)`).

11.5 Regularization and stability tools

- **Dropout:** randomly switch off a fraction of neurons during training, so the network can't rely on any one. Off at evaluation time (`model.eval()`).
- **Weight decay:** L2-style penalty on weights (Chapter 8).
- **Early stopping:** stop when validation loss stops improving; keep the best checkpoint.
- **Data augmentation:** create varied copies of training data (flipped, cropped, rotated images).
- **Batch normalization / layer normalization:** rescale activations inside the network to keep training stable. Transformers use layer norm.
- **Mixed precision:** compute in 16-bit numbers for about 2× speed and half the memory (`torch.autocast`).

11.6 Reading loss curves

Always plot training loss and validation loss per epoch.

What you see	Meaning	Action
Both falling	Still learning	Train longer
Train falling, validation rising	Overfitting	Stop early, add regularization or data
Both flat and high	Underfitting or learning rate too low	Bigger model, higher learning rate
Loss jumps to NaN	Learning rate too high, or bad data	Lower the rate, clip gradients, check inputs

Try it

1. Record `val_loss` each epoch in a list, train for 20 epochs, and plot it. When does overfitting start?
2. Swap AdamW for `torch.optim.SGD(model.parameters(), lr=0.01)`. How does it compare?
3. Remove the dropout layers and compare training vs validation accuracy.

Chapter 12: Convolutional networks and transfer learning

A **convolutional neural network (CNN)** is built for images. Instead of treating every pixel as unrelated, it slides small pattern detectors across the image, so a feature learned in one corner is recognized everywhere.

12.1 Why flattening images is wasteful

Chapter 11's network flattened each 28×28 image into 784 numbers and lost the idea that neighbouring pixels belong together. A 1000×1000 colour photo would need 3 million inputs per neuron — far too many weights.

12.2 The building blocks

- **Convolution:** a small grid of weights (a **filter** or **kernel**, often 3×3) slides over the image. At each position it computes a weighted sum, producing a **feature map** that lights up where its pattern appears (a vertical edge, a curve, later an eye). The same filter weights are reused at every position — that's what keeps CNNs small.
- **Channels:** a colour image has 3 (red, green, blue). Each conv layer outputs many channels, one per filter.
- **Stride and padding:** stride is how far the filter jumps each step; padding adds a border so edges aren't lost.
- **Pooling:** shrink each feature map, e.g. **max pooling** keeps the largest value in every 2×2 block. Makes the network cheaper and less sensitive to small shifts.
- **Fully connected head:** at the end, flatten and use ordinary layers to classify.

Stacked, these learn a hierarchy: edges → textures → parts → objects.

12.3 A CNN in PyTorch

Replace the model in Chapter 11's script with this; everything else stays the same.

```
import torch.nn as nn

model = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=3, padding=1), nn.ReLU(),      # 1 input channel -> 32
    feature maps, 28x28                                       # -> 14x14
    nn.MaxPool2d(2),                                           # -> 7x7
    nn.Conv2d(32, 64, kernel_size=3, padding=1), nn.ReLU(),   # 64 * 7 * 7 = 3136
    nn.MaxPool2d(2),                                           # 64 * 7 * 7 = 3136
    nn.Flatten(),                                              # 64 * 7 * 7 = 3136
    nn.Linear(64 * 7 * 7, 128), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, 10),
).to(device)
```

Expect about 91–92% after 5 epochs — better than the plain network, with fewer weights in the early layers.

12.4 Famous architectures

Model	Year	Contribution
-------	------	--------------

Model	Year	Contribution
LeNet	1998	Early CNN for reading handwritten digits
AlexNet	2012	Won ImageNet by a huge margin on GPUs; started the deep learning boom
VGG	2014	Showed deeper stacks of small 3×3 filters work well
ResNet	2015	Residual connections made 100+ layer networks trainable
Vision Transformer (ViT)	2020	Applied transformers to images by cutting them into patches

12.5 Transfer learning: don't start from zero

Training a big image model needs millions of images. **Transfer learning** takes a model already trained on a huge dataset (ImageNet, 1.2 million images) and adapts it to your task with far less data. The early layers already know edges and textures; you only retrain the last part.

```
import torch, torch.nn as nn
from torchvision import models

model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT) # pre-trained on ImageNet
for p in model.parameters():
    p.requires_grad = False # freeze everything
model.fc = nn.Linear(model.fc.in_features, 3) # new head: e.g. 3 of your classes
optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3) # train only the head
```

Feed it your own images with `torchvision.datasets.ImageFolder("my_images", transform=models.ResNet18_Weights.DEFAULT.transforms())`, where `my_images` has one sub-folder per class. With a few hundred images per class you can often get strong accuracy.

Fine-tuning goes one step further: after training the head, unfreeze some or all layers and continue with a small learning rate. The same pattern — pre-train big once, adapt cheaply many times — is exactly how LLMs are used (Chapter 16).

12.6 Other vision tasks

- **Object detection:** find and box every object (YOLO, Faster R-CNN).
- **Segmentation:** label every pixel (U-Net, Mask R-CNN, Segment Anything).
- **Image generation:** create new images (Chapter 19).

Try it

1. Add data augmentation: `transforms.Compose([transforms.RandomHorizontalFlip(), transforms.ToTensor()])` for the training set only.
2. Count parameters of both models: `sum(p.numel() for p in model.parameters())`.
3. Make a folder with photos of two kinds of objects from your phone and fine-tune ResNet-18 on them.

Chapter 13: Sequences — RNNs, LSTMs and their limits

Text, speech, stock prices and sensor readings are **sequences**: order matters. Before transformers, **recurrent neural networks (RNNs)** were the main tool for them. Understanding their limits explains why transformers won.

13.1 The recurrent idea

An RNN reads a sequence one step at a time and carries a **hidden state** — a memory vector — from each step to the next:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

At word t , the new memory h_t mixes the current word x_t with the previous memory h_{t-1} . The same weights are used at every step, so it handles sequences of any length.

13.2 The problem: forgetting

In a long sentence, information from early words must pass through dozens of steps. During training, gradients shrink at every step on the way back (the **vanishing gradient** problem from Chapter 10). The network struggles to connect "The keys that I left on the kitchen table yesterday **are...**" back to "keys".

13.3 LSTMs and GRUs

The **Long Short-Term Memory (LSTM)** network (1997) adds a separate **cell state** and three **gates** — small learned switches:

- **Forget gate**: what to erase from memory.
- **Input gate**: what new information to store.
- **Output gate**: what to reveal at this step.

The **GRU** is a simpler version with two gates. Both remember far longer than plain RNNs, and they powered machine translation, speech recognition and phone keyboards through the 2010s.

13.4 An LSTM in PyTorch

A sentiment classifier skeleton: token ids go in, positive/negative comes out.

```
import torch, torch.nn as nn

class SentimentLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim=64, hidden_dim=128):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)  # word id -> vector
```

```

(Chapter 14)
self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
self.head = nn.Linear(hidden_dim, 2)

def forward(self, token_ids):                # shape: (batch, sequence_length)
    x = self.embed(token_ids)
    _, (h_last, _) = self.lstm(x)            # final hidden state summarizes
the sentence                                the sentence
    return self.head(h_last[-1])

model = SentimentLSTM(vocab_size=10_000)
fake_batch = torch.randint(0, 10_000, (8, 50)) # 8 sentences of 50 tokens
print(model(fake_batch).shape)                 # torch.Size([8, 2])
Train it with the same loop as Chapter 11.

```

13.5 Sequence-to-sequence and the birth of attention

For translation, an **encoder** RNN read the whole source sentence into one vector and a **decoder** RNN wrote the translation from it. Squeezing a whole sentence into one vector was a bottleneck.

In 2014, researchers added **attention**: at each output word, the decoder could look back at *all* encoder states and focus on the relevant ones. Translation quality jumped.

Then came the key question: if attention is doing the important work, do we need the slow, step-by-step recurrence at all? The 2017 answer — no — is the transformer (Chapter 15).

13.6 Why transformers replaced RNNs

	RNN / LSTM	Transformer
Reads a sequence	One step at a time	All positions at once
Training speed on GPUs	Slow: steps can't run in parallel	Fast: fully parallel
Long-range connections	Pass through many steps; fade	Direct: any word attends to any word
Cost as sequences grow	Grows linearly	Grows with the square of the length

RNNs are still useful for small, streaming or on-device problems, and newer **state-space models** (such as Mamba) revive the recurrent idea for very long sequences.

13.7 Time series

Forecasting (sales, energy demand) is a sequence task too. Start with simple baselines ("same as last week"), then classical methods (ARIMA, exponential smoothing) or gradient boosting on lag features, before trying LSTMs or transformers. Always split time series by date, never randomly.

Try it

1. Replace `nn.LSTM` with `nn.GRU`. What else in `forward` must change? (Hint: a GRU returns only `h`, not `(h, c)`.)

2. Count the parameters of the model. Which layer has the most?
3. Write down in your own words why a transformer can train faster than an LSTM.

Part 4 — Generative AI and transformers

Chapter 14: Tokens and embeddings

Neural networks only understand numbers. Before an LLM can read text, it chops it into **tokens** and turns each token into a list of numbers, an **embedding**, whose position in space captures meaning.

14.1 Tokenization

A **tokenizer** splits text into pieces from a fixed **vocabulary** (typically 30,000–200,000 pieces) and maps each piece to an integer id.

- Split by **characters**: tiny vocabulary, but very long sequences.
- Split by **words**: short sequences, but a huge vocabulary and no way to handle new words.
- Split into **subwords** (the modern choice): common words stay whole, rare words break into pieces. "unbelievably" → "un", "believ", "ably".

The most common algorithm is **byte-pair encoding (BPE)**: start from single characters, then repeatedly merge the most frequent adjacent pair into a new token, until the vocabulary reaches the target size.

```
# pip install tiktoken
import tiktoken

enc = tiktoken.get_encoding("cl100k_base")      # a GPT-4-era tokenizer
ids = enc.encode("Transformers are unbelievably useful!")
print(ids)                                     # a list of integers
print([enc.decode([i]) for i in ids])          # the pieces each integer stands for
print(len(ids), "tokens")
```

Why it matters in practice: models have a **context window** measured in tokens, API pricing is per token, and a rough rule for English is 1 token $\approx$ $\frac{3}{4}$ of a word. Languages other than English, and code, often need more tokens per word.

14.2 From ids to vectors: embeddings

An integer id carries no meaning: token 1045 isn't "closer" to 1046. So the model has an **embedding table** — a learned matrix with one row per token. Looking up a token gives its vector, for example 768 or 4,096 numbers.

```
import torch.nn as nn
embedding = nn.Embedding(num_embeddings=50_000, embedding_dim=768)  # learned during training
```

During training, tokens used in similar contexts drift to similar vectors. Distance in this space becomes a measure of meaning.

14.3 The famous example: word2vec

In 2013, **word2vec** learned word vectors by predicting a word from its neighbours. The vectors captured relationships as directions:

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$$

The "male → female" direction is roughly the same everywhere in the space, as are "country → capital" and "present → past tense".

14.4 Contextual embeddings

word2vec gives "bank" one vector whether it's a river bank or a savings bank. Transformers fix this: the token's embedding is only the starting point, and each attention layer updates it using the surrounding words. By the last layer, "bank" near "river" and "bank" near "loan" have very different vectors.

14.5 Sentence embeddings and semantic search

An **embedding model** turns a whole sentence or document into one vector. Texts with similar meaning land close together, even with no words in common. We measure closeness with **cosine similarity** (1 = same direction, 0 = unrelated).

```
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer("all-MiniLM-L6-v2")
sentences = [
    "How do I reset my password?",
    "I forgot my login credentials.",
    "What's the weather in Ottawa today?",
]
vectors = model.encode(sentences)
print(vectors.shape)                # (3, 384): 384 numbers per sentence
print(util.cos_sim(vectors, vectors)) # sentences 1 and 2 score far higher
                                     # than 1 and 3
```

This is the core of **semantic search**, recommendation, clustering documents, and **RAG** (Chapter 18). Collections of embeddings are stored in **vector databases** (FAISS, Chroma, pgvector, Pinecone) that can find the nearest vectors among millions in milliseconds.

14.6 Position

Attention on its own sees a bag of tokens with no order: "dog bites man" and "man bites dog" look the same. So transformers add **positional information** to each embedding. The original transformer added fixed sine-wave patterns; most modern LLMs use **rotary position embeddings (RoPE)**, which rotate vectors by an angle that depends on position.

Try it

1. Tokenize a sentence in English and the same sentence in Persian or French. Which uses more tokens?
2. Add five sentences of your own to the embedding example, including two paraphrases. Do the paraphrases score highest?

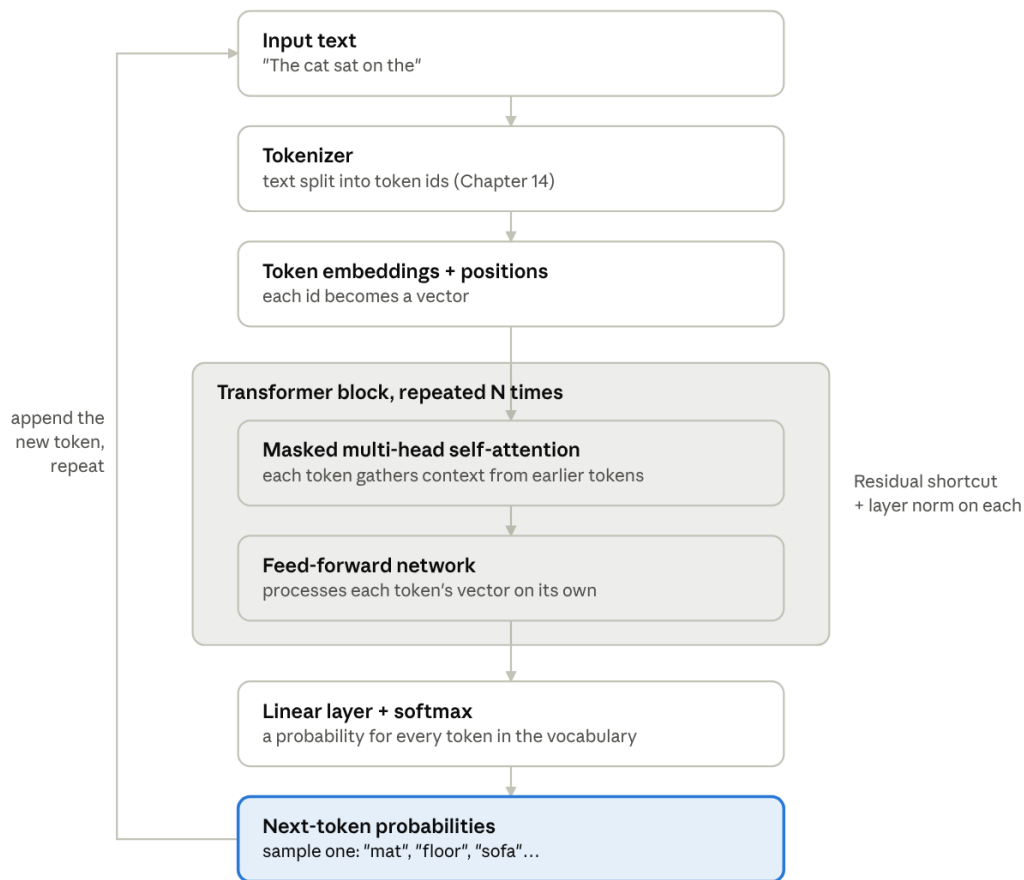
3. Build a tiny search engine: embed 10 FAQ answers, embed a question, and print the answer with the highest cosine similarity.

Chapter 15: Attention and the transformer

A transformer is a stack of identical blocks, each doing two things: **attention** (tokens share information with each other) and a **feed-forward network** (each token thinks on its own). This chapter explains both, then you build a working mini-GPT.

15.1 The whole architecture at a glance

A GPT model predicts one token, then feeds it back in



Decoder-only transformer (GPT) · one prediction step

Text becomes tokens, tokens become vectors, the vectors pass through N transformer blocks, and the final layer scores every token in the vocabulary. The model samples one, appends it, and runs again.

15.2 Attention: queries, keys and values

Think of a library. You arrive with a **query** ("books about cats"). Every book has a **key** on its spine (its topic). You compare your query with every key, and take the **values** (contents) of the books that match best.

In self-attention, every token produces all three, by multiplying its vector by three learned weight matrices:

- **Query (Q)**: what am I looking for?
- **Key (K)**: what do I contain?
- **Value (V)**: what information will I pass on if chosen?

Each token's query is compared (dot product) with every token's key. High scores mean "relevant". Softmax turns the scores into weights summing to 1, and the token's new vector is the weighted mix of all the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Dividing by $\sqrt{d_k}$ (the key length) keeps the scores from getting too large, which would make softmax too spiky to train. In "The animal didn't cross the street because it was too tired", the query from "it" matches the key from "animal" strongly, so "it" absorbs information about the animal.

15.3 Masking, heads and the rest of the block

- **Causal mask**: a GPT must predict the next token without seeing it, so each token may only attend to itself and earlier tokens. Future positions get a score of $-\infty$ before softmax, which becomes a weight of 0.
- **Multi-head attention**: run several attentions in parallel (e.g. 12 heads), each with its own Q, K, V. One head might track grammar, another who "it" refers to. Their outputs are joined and mixed.
- **Feed-forward network**: two linear layers with a GELU in between, applied to each token separately. Much of the model's factual knowledge seems to be stored here.
- **Residual connections**: each part's output is *added* to its input ($x + \text{attention}(x)$), so information and gradients can flow straight through 100+ layers.
- **Layer normalization**: rescales each vector to keep numbers stable.

15.4 Three families of transformers

Family	Attention	Trained to	Examples	Best at
Encoder-only	Sees the whole text both ways	Fill in masked words	BERT, RoBERTa	Classification, search embeddings
Decoder-only	Causal: only earlier tokens	Predict the next token	GPT, Claude, Llama, Mistral	Generating text, chat, code

Family	Attention	Trained to	Examples	Best at
Encoder–decoder	Encoder reads, decoder writes	Map input text to output text	Original Transformer, T5	Translation, summarization

Nearly all modern chat LLMs are decoder-only.

15.5 Build a tiny GPT

This is a complete, character-level GPT trained on Shakespeare's plays, about 1.8 million parameters. It's a simplified version of Andrej Karpathy's well-known *nanoGPT*. Use a GPU runtime in Colab; training takes a few minutes.

```
import torch, torch.nn as nn, torch.nn.functional as F
import urllib.request

# ---- data: characters are our tokens ----
url = "https://raw.githubusercontent.com/karpathy/char-
rnn/master/data/tinyshakespeare/input.txt"
text = urllib.request.urlopen(url).read().decode("utf-8")
chars = sorted(set(text)); vocab_size = len(chars)
stoi = {c: i for i, c in enumerate(chars)}; itos = {i: c for c, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data)); train_data, val_data = data[:n], data[n:]

device = "cuda" if torch.cuda.is_available() else "cpu"
block_size, batch_size = 128, 64          # context length, sequences per batch
n_embd, n_head, n_layer, dropout = 192, 6, 4, 0.1

def get_batch(split):
    d = train_data if split == "train" else val_data
    ix = torch.randint(len(d) - block_size - 1, (batch_size,))
    x = torch.stack([d[i:i + block_size] for i in ix])
    y = torch.stack([d[i + 1:i + block_size + 1] for i in ix])  # targets: shifted
    by one
    return x.to(device), y.to(device)

# ---- the model ----
class CausalSelfAttention(nn.Module):
    def __init__(self):
        super().__init__()
        self.qkv = nn.Linear(n_embd, 3 * n_embd)  # makes Q, K, V in one go
        self.proj = nn.Linear(n_embd, n_embd)
        self.drop = nn.Dropout(dropout)
        self.register_buffer("mask", torch.tril(torch.ones(block_size, block_size)))

    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(n_embd, dim=2)
        # split into heads: (B, heads, T, head_size)
        q = q.view(B, T, n_head, C // n_head).transpose(1, 2)
        k = k.view(B, T, n_head, C // n_head).transpose(1, 2)
```

```

        v = v.view(B, T, n_head, C // n_head).transpose(1, 2)
        att = (q @ k.transpose(-2, -1)) / (k.size(-1) ** 0.5)      # scores
        att = att.masked_fill(self.mask[:T, :T] == 0, float("-inf")) # no peeking
ahead
        att = self.drop(F.softmax(att, dim=-1))                    # weights
        out = (att @ v).transpose(1, 2).contiguous().view(B, T, C) # mix values,
join heads
        return self.drop(self.proj(out))

class Block(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1, self.attn = nn.LayerNorm(n_embd), CausalSelfAttention()
        self.ln2 = nn.LayerNorm(n_embd)
        self.ffn = nn.Sequential(nn.Linear(n_embd, 4 * n_embd), nn.GELU(),
                                  nn.Linear(4 * n_embd, n_embd), nn.Dropout(dropout))

    def forward(self, x):
        x = x + self.attn(self.ln1(x))    # residual connection
        x = x + self.ffn(self.ln2(x))    # residual connection
        return x

class TinyGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok = nn.Embedding(vocab_size, n_embd)
        self.pos = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block() for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        x = self.tok(idx) + self.pos(torch.arange(T, device=idx.device))
        logits = self.head(self.ln_f(self.blocks(x)))
        loss = None if targets is None else F.cross_entropy(logits.view(-1,
vocab_size), targets.view(-1))
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0):
        for _ in range(max_new_tokens):
            logits, _ = self(idx[:, -block_size:])
            probs = F.softmax(logits[:, -1, :] / temperature, dim=-1)
            idx = torch.cat([idx, torch.multinomial(probs, 1)], dim=1) # sample,
append, repeat
        return idx

# ---- training: the same loop as Chapter 11 ----
model = TinyGPT().to(device)
print(sum(p.numel() for p in model.parameters()) / 1e6, "million parameters")
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
for step in range(3000):
    x, y = get_batch("train")
    _, loss = model(x, y)

```

```

opt.zero_grad(); loss.backward(); opt.step()
if step % 500 == 0:
    model.eval()
    with torch.no_grad():
        _, val_loss = model(*get_batch("val"))
    model.train()
    print(f"step {step}: train loss {loss.item():.2f}, val loss {val_loss.item():.2f}")

start = torch.zeros((1, 1), dtype=torch.long, device=device) # start from a newline
print(decode(model.generate(start, 500)[0].tolist()))

```

The loss starts around 4.2 (random guessing among ~65 characters) and falls to roughly 1.5. The output won't make sense, but it will look like a play: character names, line breaks, old-English-sounding words. You built a language model from nothing. Real LLMs use the same architecture with subword tokens, thousands of dimensions, dozens of layers and trillions of training tokens.

15.6 Sampling: how the next token is chosen

- **Greedy**: always take the most likely token. Repetitive.
- **Temperature**: divide scores before softmax. Below 1 is safer and more focused; above 1 is more creative and more random.
- **Top-k**: sample only from the k most likely tokens.
- **Top-p (nucleus)**: sample from the smallest set of tokens whose probabilities add up to p (e.g. 0.9).

15.7 Scaling and efficiency

Researchers found **scaling laws**: loss falls predictably as you increase parameters, data and compute together. That's why models grew from millions to hundreds of billions of parameters. Tricks that make big models practical include **FlashAttention** (faster attention on GPUs), the **KV cache** (reuse past keys and values when generating), **mixture of experts** (only part of the network runs for each token), and **quantization** (storing weights in 8 or 4 bits).

Try it

1. Generate with `temperature=0.5` and `temperature=1.5`. Describe the difference.
2. Double `n_layer` to 8. Does validation loss improve? How much slower is training?
3. Remove the causal mask line and train again. Why does the training loss drop suspiciously fast, and why is the model useless for generation?

Chapter 16: Training LLMs

An assistant like ChatGPT or Claude is made in two big phases. **Pre-training** teaches a model language and knowledge by predicting the next token on trillions of tokens. **Post-training** turns that raw text predictor into a helpful, honest, safe assistant.

16.1 The pipeline

Stage	Data	What the model learns	Rough cost
-------	------	-----------------------	------------

Stage	Data	What the model learns	Rough cost
1. Pre-training	Trillions of tokens of web pages, books, code, papers	Grammar, facts, reasoning patterns; becomes a base model	Weeks to months on thousands of GPUs
2. Supervised fine-tuning (SFT)	Thousands to millions of example conversations written or checked by people	To follow instructions and answer as an assistant	Hours to days
3. Preference tuning (RLHF, DPO, RLAIIF)	Pairs of answers ranked better/worse	Which answers people prefer: helpful, harmless, honest	Days
4. RL on verifiable tasks	Maths, code and puzzles with checkable answers	Longer step-by-step reasoning ("reasoning models")	Days to weeks
5. Evaluation and red-teaming	Benchmarks, human raters, adversarial testers	Nothing — it checks the model (Chapter 17)	Ongoing

16.2 Pre-training

Pre-training is exactly what you did with the tiny GPT in Chapter 15, at vast scale. The loss is cross-entropy on the next token: how surprised the model was by the real continuation.

$$L = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_1, \dots, x_{t-1})$$

It's called **self-supervised**: the text labels itself, so no human annotation is needed. What matters most:

- **Data quality**: deduplication, removing spam and toxic content, filtering by quality, and a careful mix of domains (web, code, maths, languages). Data work is often the biggest differentiator between labs.
- **Scale**: a common rule of thumb from the *Chinchilla* paper is about 20 training tokens per parameter for compute-efficient training; modern models are often trained on far more. Training compute is roughly $6 \times \text{parameters} \times \text{tokens}$ floating-point operations.
- **Distributed training**: no single GPU can hold a large model. **Data parallelism** gives each GPU a different batch; **tensor** and **pipeline parallelism** split the model itself across GPUs; **FSDP**/**ZeRO** shards weights and optimizer state to save memory.
- **Stability**: warmup, learning-rate decay, gradient clipping, mixed precision and frequent checkpoints (Chapter 11), because a crash three weeks in is very expensive.

The result is a **base model**: it continues text brilliantly, but if you ask it a question it may just write more questions.

16.3 Supervised fine-tuning (SFT)

SFT continues training on example conversations: a prompt and an ideal response. The loss is the same next-token cross-entropy, usually computed only on the response tokens. Conversations are formatted with a **chat template**, special tokens marking who is speaking (system, user, assistant). Quality beats quantity: a few thousand excellent examples can transform a base model's behaviour.

16.4 Preference tuning: RLHF, DPO and RLAI

Some qualities are easier to judge than to write: which of two answers is clearer, kinder, or more accurate?

- **RLHF** (reinforcement learning from human feedback): people rank pairs of answers; a **reward model** is trained to predict their preferences; then the LLM is optimized with reinforcement learning (classically **PPO**) to score highly, with a penalty for drifting too far from the SFT model.
- **DPO** (direct preference optimization): skips the separate reward model and RL loop, and adjusts the LLM directly from preferred/rejected pairs. Simpler and widely used.
- **RLAI / Constitutional AI**: an AI model, guided by a written set of principles, produces some of the feedback instead of humans. Anthropic uses this approach for Claude.

16.5 Reasoning models

Newer models are further trained with reinforcement learning on problems whose answers can be checked automatically (a maths result, code that passes tests). Rewarding correct final answers teaches the model to "think" longer, write out steps, and check its own work before answering.

16.6 Fine-tuning an LLM yourself

You won't pre-train an LLM, but you can fine-tune an open model on your own data. First ask whether you need to:

Approach	Use when	Cost
Prompting (Chapter 18)	Always try first	Free
RAG (Chapter 18)	The model needs your facts or documents	Low
Fine-tuning	You need a consistent style, format or narrow skill, or a smaller, cheaper model that matches a big one	Medium
Pre-training from scratch	A new language or domain with no suitable model; big budget	Very high

Full fine-tuning updates every weight — memory-hungry. **Parameter-efficient fine-tuning (PEFT)** freezes the model and trains a tiny add-on. The most popular is **LoRA** (low-rank adaptation): it adds small matrices beside the existing weights and trains only those, often under 1% of the parameters.

QLoRA also loads the frozen model in 4-bit, so a 7-billion-parameter model can be fine-tuned on one consumer GPU.

16.7 Code: LoRA fine-tuning with Hugging Face

This fine-tunes a small open model (0.5 billion parameters, fits on Colab's free GPU) on 2,000 example conversations. Hugging Face libraries change quickly; if an argument errors, check the current [TRL documentation](#).

```
# pip install -U transformers datasets peft trl accelerate
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, pipeline
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig

model_name = "Qwen/Qwen2.5-0.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)

# Conversations stored as {"messages": [{"role": "user", ...}, {"role": "assistant", ...}]}
dataset = load_dataset("trl-lib/Capybara", split="train[:2000]")

lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                  target_modules="all-linear", task_type="CAUSAL_LM")
args = SFTConfig(output_dir="qwen-lora", num_train_epochs=1,
                 per_device_train_batch_size=4, gradient_accumulation_steps=4,
                 learning_rate=2e-4, logging_steps=20)

trainer = SFTTrainer(model=model, args=args, train_dataset=dataset,
                    peft_config=lora, processing_class=tokenizer)
trainer.train()          # watch the training loss fall in the logs
trainer.save_model()     # saves only the small LoRA adapter

chat = pipeline("text-generation", model=trainer.model, tokenizer=tokenizer)
print(chat([{"role": "user", "content": "Explain overfitting in two sentences."}],
           max_new_tokens=100)[0]["generated_text"])
```

To fine-tune on your own data, replace the dataset with a list of your conversations in the same messages format: `Dataset.from_list(my_conversations)`. For preference tuning, TRL's `DPOTrainer` takes `prompt`, `chosen` and `rejected` columns in the same style.

16.8 Practical tips

- **Data first:** 500 clean, consistent examples beat 50,000 messy ones. Read your data.
- **Hold out an evaluation set** and compare against the untuned model (Chapter 17). Fine-tuning can make a model worse at things you didn't train on (**catastrophic forgetting**).
- **Typical LoRA settings:** rank 8–64, learning rate 1e-4 to 3e-4, 1–3 epochs. More epochs on small data overfits fast.
- **Track experiments** with a tool like Weights & Biases or MLflow (`report_to="wandb"`).

Try it

1. Before training, ask the base model a question; ask again after. What changed?

2. Create 50 conversations in a particular style (for example, answers that always end with a one-line summary) and fine-tune on them.
3. Change the LoRA rank r from 16 to 4. Compare training loss and the adapter's file size.

Chapter 17: Evaluating LLMs

Evaluating an LLM is harder than evaluating a classifier, because there is rarely one correct answer. Good LLM evaluation combines several layers: automatic metrics, public benchmarks, your own task-specific test set, an LLM judge calibrated against humans, and monitoring in production.

17.1 The layers of LLM evaluation

Layer	Question	Tools	Weakness
Intrinsic	How well does it model language?	Perplexity, validation loss	Says little about usefulness
Public benchmarks	How does it compare with other models?	MMLU-style knowledge tests, GSM8K-style maths, HumanEval-style coding, leaderboards	Contamination, saturation, not your task
Task evals	Does it do <i>my</i> job well?	Your own test set + reference answers + metrics	Needs effort to build
LLM-as-judge	Is this open-ended answer good?	A strong model grading with a rubric	Judge bias; must be validated
Human evaluation	Do people actually find it good?	Expert raters, side-by-side comparisons	Slow, expensive, raters disagree
Safety evals	Can it be misused or cause harm?	Red-teaming, jailbreak suites, bias tests	Never complete
Online	Does it work for real users?	A/B tests, thumbs up/down, task completion, monitoring	Only after launch

17.2 Perplexity

Perplexity is e raised to the average next-token loss. Intuitively: how many tokens the model is "choosing between" at each step. Lower means the text is less surprising to the model.

```
import math, torch
from transformers import AutoTokenizer, AutoModelForCausalLM

tok = AutoTokenizer.from_pretrained("gpt2")
lm = AutoModelForCausalLM.from_pretrained("gpt2")
```

```
def perplexity(text):
    ids = tok(text, return_tensors="pt").input_ids
    with torch.no_grad():
        loss = lm(ids, labels=ids).loss      # average next-token cross-entropy
    return math.exp(loss.item())

print(perplexity("The cat sat on the mat. ")) # low
print(perplexity("Mat the on sat cat the. ")) # much higher
```

Use it to compare checkpoints of the same model during training, not models with different tokenizers.

17.3 Reference-based metrics

When you have a correct answer to compare with:

Metric	Measures	Good for
Exact match / accuracy	Output equals the answer (after normalizing)	Multiple choice, short factual answers, classification
Token F1	Word overlap with the answer	Extractive question answering
BLEU / ROUGE	n-gram overlap with a reference	Translation / summarization; weak for modern LLMs
BERTScore / embedding similarity	Similarity of meaning, via embeddings	Paraphrased answers
pass@k	Share of problems where at least one of k generated programs passes the unit tests	Code generation
JSON / schema validity	Output parses and matches the required format	Structured outputs, tool calls

17.4 Benchmarks and their traps

Public benchmarks cover knowledge, maths, coding, reasoning, long context and tool use. Use them to shortlist models, then test on your own task. Watch out for:

- **Contamination:** benchmark questions leaked into training data, inflating scores.
- **Saturation:** when top models all score above 90%, the benchmark stops separating them.
- **Prompt sensitivity:** small wording or formatting changes can move scores several points. Keep prompts fixed when comparing.

17.5 Build your own evaluation set

This is the most valuable evaluation you'll ever do.

1. Collect 50–500 realistic inputs, ideally from real users or logs.
2. Cover the common cases, known edge cases, and inputs that should be refused or escalated.
3. Write a reference answer or a clear grading rubric for each.

4. Split into a **development set** (to iterate prompts on) and a **held-out set** (for final comparisons), just like Chapter 4.
5. Version it with your code (Chapter 21) and run it on every prompt or model change, like unit tests. A drop is a **regression**.
6. Add every real failure you find in production as a new test case.

17.6 LLM-as-judge

For open-ended outputs, a strong LLM can grade answers against a rubric. Scores are cheap and fast, but only trustworthy once you've checked them against human ratings.

```
# pip install anthropic (set the ANTHROPIC_API_KEY environment variable)
import json, anthropic
import pandas as pd

client = anthropic.Anthropic()

JUDGE_PROMPT = """You are grading an AI assistant's answer.
Question: {question}
Reference answer: {reference}
Assistant's answer: {answer}

Score the answer from 1 to 5 for correctness against the reference
(5 = fully correct and complete, 1 = wrong or missing). Explain first, then score.
Respond ONLY with JSON: {"reasoning": "...", "score": <1-5>}}"""

def judge(question, reference, answer):
    msg = client.messages.create(
        model="claude-sonnet-5", max_tokens=400,
        messages=[{"role": "user", "content": JUDGE_PROMPT.format(
            question=question, reference=reference, answer=answer)}])
    return json.loads(msg.content[0].text)

eval_set = [
    {"question": "What does 'overfitting' mean?",
     "reference": "The model memorizes training data, including noise, and performs poorly on new data."},
    {"question": "Which metric matters most for cancer screening, precision or recall?",
     "reference": "Recall, because missing a real case is the costliest error."},
]

def run_eval(generate):
    # generate: any function question -> answer
    rows = []
    for case in eval_set:
        answer = generate(case["question"])
        rows.append(**case, "answer": answer, **judge(case["question"],
case["reference"], answer))
    df = pd.DataFrame(rows)
    print("mean score:", df["score"].mean())
    return df
```

Known judge biases and how to reduce them:

- **Position bias** in pairwise comparisons (preferring the first answer): run each pair twice with the order swapped.
- **Verbosity bias** (preferring longer answers): tell the rubric that length is not quality.
- **Self-preference** (a model favouring its own outputs): use a judge from a different model family when possible.
- **Vague rubrics**: use specific, observable criteria and a small scale (pass/fail or 1–5), and ask for reasoning before the score.
- **Validate**: have humans grade 50–100 of the same answers and measure agreement (percentage agreement, or Cohen's kappa). Improve the judge prompt until agreement is close to human–human agreement.

17.7 Evaluating RAG systems

A RAG system (Chapter 18) can fail at retrieval or at generation, so measure both:

Part	Metric	Question
Retrieval	Recall@k	Is the right document among the top k results?
Retrieval	MRR (mean reciprocal rank)	How high does the first right document appear?
Generation	Faithfulness / groundedness	Is every claim supported by the retrieved text?
Generation	Answer relevance	Does it actually answer the question?
End to end	Correctness	Does it match the reference answer?

Libraries such as Ragas and DeepEval implement many of these with LLM judges.

17.8 Safety and robustness

- **Red-teaming**: people (and other models) actively try to make the system misbehave: harmful instructions, leaking private data, **prompt injection** hidden in documents or web pages.
- **Bias tests**: swap names, genders or dialects in otherwise identical prompts and compare the outputs.
- **Hallucination checks**: questions with known answers, and questions with no answer where "I don't know" is correct.
- **Robustness**: typos, rephrasings, other languages, very long inputs.

17.9 Online evaluation

Offline tests don't capture everything. After launch, run A/B tests on real traffic, log user feedback, track task-completion and escalation rates, sample conversations for human review, and watch latency and cost per request.

Try it

1. Write 20 test questions from your own work, with reference answers, and run two different models through `run_eval`.
2. Grade the same 20 answers yourself. How often do you agree with the judge?
3. Rerun the judge with a pairwise prompt ("Which answer is better, A or B?") and swap the order. Does the verdict change?

Chapter 18: Using LLMs — prompting, APIs, RAG and agents

Most AI products today aren't new models; they're systems built around an existing LLM. The three core techniques are **prompting** (asking well), **RAG** (giving the model your information) and **tool use** (letting it take actions).

18.1 Calling an LLM from code

```
# pip install anthropic (set ANTHROPIC_API_KEY; get a key at console.anthropic.com)
import anthropic

client = anthropic.Anthropic()
msg = client.messages.create(
    model="claude-sonnet-5",
    max_tokens=500,
    system="You are a patient tutor who explains AI to beginners.",
    messages=[{"role": "user", "content": "What is a transformer, in three sentences?"}],
)
print(msg.content[0].text)
```

Key ideas: the **system prompt** sets the role and rules; **messages** hold the conversation (the model has no memory — you resend the history each call); **max_tokens** caps the answer length; you pay per input and output token. Other providers (OpenAI, Google, open models via Hugging Face or Ollama) work almost identically.

18.2 Prompting that works

Technique	What it means	Example
Be clear and specific	State the task, audience, format and length	"Summarize for a busy manager in 3 bullets, max 15 words each"
Give context	Explain why and for whom	"This goes to new drivers who have never used the app"
Show examples (few-shot)	2–5 input/output examples of what good looks like	Three labelled reviews, then the new one
Structure with tags	Separate instructions from data	<document>...</document> then the question
Let it think	Ask for step-by-step reasoning before the final answer	"Work through it step by step, then give the answer in

Technique	What it means	Example
		<answer> tags"
Specify the output format	JSON with named fields, for code to parse	"Return only JSON: {"sentiment": ..., "confidence": ...}"
Allow "I don't know"	Reduces invented answers	"If the document doesn't say, reply 'not stated'"
Iterate with evals	Change one thing, rerun your eval set (Chapter 17)	Prompt v1 vs v2 on 50 cases

18.3 Why RAG?

An LLM only knows its training data: nothing after its cutoff date and nothing private to you. It may also **hallucinate** — produce fluent, confident, wrong answers. **Retrieval-augmented generation (RAG)** fixes both: find the relevant passages from your documents, paste them into the prompt, and instruct the model to answer only from them, citing sources.

The RAG pipeline:

1. **Ingest**: load documents and split them into **chunks** (e.g. 200–500 words, with some overlap).
2. **Embed**: turn each chunk into a vector (Chapter 14) and store it in a vector database.
3. **Retrieve**: embed the user's question and fetch the most similar chunks.
4. **Generate**: give the LLM the question plus the retrieved chunks.

18.4 A minimal RAG system

```
# pip install sentence-transformers anthropic
import numpy as np, anthropic
from sentence_transformers import SentenceTransformer

docs = [
    "Employees get 15 vacation days per year, rising to 20 after five years.",
    "Expense reports must be submitted within 30 days with receipts attached.",
    "Remote work is allowed up to three days per week with manager approval.",
    "The office is closed on statutory holidays.",
]

embedder = SentenceTransformer("all-MiniLM-L6-v2")
doc_vecs = embedder.encode(docs, normalize_embeddings=True)

def retrieve(question, k=2):
    q = embedder.encode([question], normalize_embeddings=True)[0]
    scores = doc_vecs @ q # cosine similarity (vectors are
normalized)
    return [docs[i] for i in np.argsort(-scores)[:k]]

def answer(question):
    context = "\n".join(f"<doc>{d}</doc>" for d in retrieve(question))
```

```

prompt = (f"Answer using ONLY the documents below. If they don't contain the
answer, "
          f"say you don't know.\n\n{context}\n\nQuestion: {question}")
msg = anthropic.Anthropic().messages.create(
    model="claude-sonnet-5", max_tokens=300,
    messages=[{"role": "user", "content": prompt}])
return msg.content[0].text

print(answer("How many vacation days do I get after six years?")) # 20
print(answer("What is the dress code?")) # should say it
doesn't know

```

Improving a RAG system, roughly in order of impact: better chunking (split on headings, keep tables whole), **hybrid search** (combine embeddings with keyword search such as BM25), a **reranker** model that re-scores the top 20–50 results, adding metadata filters (date, department), and evaluating retrieval and answers separately (Chapter 17.7).

18.5 Tool use and function calling

You can describe functions to the model — "get_weather(city)", "search_orders(customer_id)". Instead of answering directly, the model replies with a structured request to call one. Your code runs the function, sends the result back, and the model uses it in its answer. This lets LLMs get live data, do exact calculations and take actions. The **Model Context Protocol (MCP)** is an open standard for connecting models to tools and data sources in a reusable way.

18.6 Agents

An **agent** is an LLM running in a loop: decide the next step, call a tool, read the result, decide again, until the goal is done. Coding assistants that edit files and run tests, and research assistants that search and read many pages, are agents.

Design tips: give agents a small set of well-described tools, limit the steps they may take, ask for human confirmation before irreversible actions (sending email, spending money, deleting data), log every step, and evaluate on complete tasks, not single answers.

18.7 Choosing a model

Consideration	Trade-off
Quality	Larger, newer models are usually more capable, but slower and pricier
Latency and cost	Small models are fast and cheap; often enough for classification or extraction
Context window	How much text fits in one request; long documents need large windows or RAG
Privacy and control	Open-weight models can run on your own servers (Llama, Mistral, Qwen)
The real test	Your own eval set (Chapter 17), not a

Consideration	Trade-off
	leaderboard

Try it

1. Write a prompt that classifies customer messages into 4 categories and returns JSON. Test it on 20 messages.
2. Replace the four RAG documents with 30 paragraphs of a real policy or manual. Ask 10 questions. Which ones fail, and is it retrieval or generation?
3. Add a retrieve score threshold: if the best match scores below 0.3, skip the LLM and reply "I don't know".

Chapter 19: Beyond text — VAEs, GANs, diffusion and multimodal models

Generating images, audio and video uses the same deep learning foundations, but different recipes. Today's image generators are mostly **diffusion models**; earlier breakthroughs came from **autoencoders** and **GANs**.

19.1 Autoencoders and VAEs

An **autoencoder** is two networks: an **encoder** squeezes an image into a small vector (the **latent** code), and a **decoder** rebuilds the image from it. Trained to reproduce its input, it learns a compressed description of the data.

A **variational autoencoder (VAE)** makes the latent space smooth and organized, so you can pick a random point in it and decode a brand-new, plausible image. VAEs produce slightly blurry images alone, but they're a key part of modern systems: Stable Diffusion runs its diffusion process inside a VAE's compact latent space.

19.2 GANs: forger versus detective

A **generative adversarial network (GAN)** (2014) trains two networks against each other:

- The **generator** turns random noise into fake images.
- The **discriminator** tries to tell real images from fakes.

As the detective improves, the forger must improve too. GANs produced the first photorealistic fake faces (StyleGAN), but training is notoriously unstable, and they can get stuck making only a few kinds of output (**mode collapse**).

19.3 Diffusion models

A **diffusion model** learns to reverse a gradual noising process:

1. **Forward process** (fixed, no learning): take a real image and add a little random noise, step by step, over e.g. 1,000 steps, until it's pure static.
2. **Training**: show the network a noisy image and the step number, and train it to predict the noise that was added. The loss is just mean squared error (Chapter 3) between the predicted and the real noise.

3. **Generation:** start from pure random noise and repeatedly subtract the predicted noise. An image gradually appears.

$$L = \|\epsilon - \epsilon_\theta(x_t, t)\|^2$$

ϵ is the real noise, ϵ_θ is the network's guess, x_t is the noisy image at step t . The network is usually a U-Net or, in newer models, a transformer (**diffusion transformer**).

19.4 Guiding with text

To make "a cat astronaut in watercolour", the prompt is turned into embeddings by a text encoder (often **CLIP**, which learned a shared space for images and captions from hundreds of millions of pairs). The denoising network attends to those embeddings through **cross-attention** at every step. **Classifier-free guidance** strengthens how closely the image follows the text.

```
# pip install diffusers transformers accelerate (needs a GPU runtime)
import torch
from diffusers import AutoPipelineForText2Image

pipe = AutoPipelineForText2Image.from_pretrained(
    "stabilityai/sd-turbo", torch_dtype=torch.float16).to("cuda")
image = pipe("a watercolour painting of a lighthouse at dawn",
             num_inference_steps=1, guidance_scale=0.0).images[0] # sd-turbo is
distilled to need 1 step
image.save("lighthouse.png")
image
```

19.5 Other modalities

Modality	Common approach	Examples of tasks
Speech to text	Transformer encoder–decoder on audio spectrograms	Transcription (Whisper)
Text to speech	Transformers or diffusion over audio tokens	Voice assistants, narration
Music	Transformers over audio tokens, diffusion	Song generation
Video	Diffusion transformers over space-time patches	Text-to-video
Vision-language	An image encoder feeding a text LLM	Describing photos, reading charts, answering questions about documents

Multimodal LLMs read images (and sometimes audio) alongside text, by turning the image into a sequence of "visual tokens" that the transformer handles like words. That's how an assistant can read a screenshot or a scanned receipt.

19.6 Comparing generative model families

Family	Strengths	Weaknesses
Autoregressive transformers	Excellent for text and code; flexible	Generate one token at a time, so long outputs are slow
VAEs	Stable training, useful latent space	Blurry outputs on their own
GANs	Very fast generation, sharp images	Unstable training, mode collapse
Diffusion	Highest image quality, diverse, controllable	Many denoising steps (distillation reduces this)

Try it

1. Generate the same prompt with three different random seeds (`generator=torch.manual_seed(n)` in the pipe call). How different are the results?
2. Change one word in the prompt at a time and observe which words have the most influence.
3. Explain in your own words why predicting noise is an easier training task than generating an image in one step.

Chapter 20: Reinforcement learning basics

Reinforcement learning (RL) learns by trial and error. Nobody gives the right answer; an **agent** acts in an **environment** and receives **rewards**, and it learns which actions lead to the most reward over time. RL taught computers to beat world champions at Go, and it's how LLMs are tuned with human preferences and trained to reason (Chapter 16).

20.1 The vocabulary

Term	Meaning	Example: a robot vacuum
Agent	The learner and decision-maker	The vacuum's controller
Environment	Everything it interacts with	The room
State	The current situation	Position, battery, dirt map
Action	A choice the agent can make	Move forward, turn, return to dock
Reward	A number saying how good the last step was	+1 per dirty patch cleaned, -10 for hitting stairs
Policy	The agent's strategy: state → action	"If battery low, return to dock"

Term	Meaning	Example: a robot vacuum
Value	Expected total future reward from a state	A room corner with lots of dirt has high value
Episode	One run from start to finish	One cleaning session

The loop: observe the state → choose an action → get a reward and a new state → learn → repeat.

20.2 Key ideas

- **Delayed reward:** the move that wins a chess game may come 30 moves before the win. The agent must learn which earlier actions deserve credit.
- **Discount factor (γ):** future rewards count a bit less than immediate ones ($\gamma = 0.99$ means a reward 100 steps away is worth about 37%).
- **Exploration vs exploitation:** try new actions to discover better options, or use the best one known so far? The common **ϵ -greedy** rule explores randomly with probability ϵ , which shrinks as the agent learns.
- **Markov decision process (MDP):** the maths framework behind RL — states, actions, transition probabilities, rewards.

20.3 Q-learning

Q-learning keeps a table $Q(\text{state}, \text{action})$ estimating the total future reward of taking an action in a state. After each step it nudges the estimate towards "the reward I just got + the best value I can reach from the new state":

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

α is the learning rate, r the reward, s' the new state.

20.4 Code: learning to cross a frozen lake

In FrozenLake, an agent walks a 4x4 grid from start to goal without falling into holes. The only reward is +1 for reaching the goal.

```
# pip install gymnasium
import numpy as np
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)
Q = np.zeros((env.observation_space.n, env.action_space.n)) # 16 states x 4 actions
alpha, gamma, epsilon = 0.1, 0.99, 1.0
rng = np.random.default_rng(0)

for episode in range(2000):
    state, _ = env.reset(seed=episode)
    done = False
    while not done:
        if rng.random() < epsilon:
            action = env.action_space.sample() # explore
        else:
```

```

        action = int(np.argmax(Q[state]))          # exploit
        new_state, reward, terminated, truncated, _ = env.step(action)
        target = reward + gamma * np.max(Q[new_state]) * (not terminated)
        Q[state, action] += alpha * (target - Q[state, action])
        state, done = new_state, terminated or truncated
        epsilon = max(0.05, epsilon * 0.995)      # explore less over time

# test the learned policy
wins = 0
for _ in range(100):
    state, _ = env.reset(); done = False
    while not done:
        state, reward, terminated, truncated, _ = env.step(int(np.argmax(Q[state])))
        done = terminated or truncated
    wins += reward
print("wins out of 100:", wins)    # 100

```

20.5 Deep RL and policy methods

A table works for 16 states, not for the pixels of a video game. **Deep Q-Networks (DQN)** replace the table with a neural network that estimates Q from the raw state — this is how DeepMind's agents learned Atari games from pixels in 2013–2015.

Policy-gradient methods learn the policy directly: increase the probability of actions that led to high reward. **PPO** (proximal policy optimization) is a stable, popular version, and it's the classic algorithm behind RLHF: the LLM is the policy, generating a response is the action, and the reward model's score is the reward. **AlphaGo** combined deep networks with tree search and self-play.

20.6 When to use RL

RL fits problems of sequential decisions with a clear reward: games, robotics, recommendation strategies, resource scheduling, and fine-tuning LLMs. It's data-hungry and hard to debug, so if you have labelled examples of the right answer, supervised learning is usually simpler.

Try it

1. Set `is_slippery=True` (the agent sometimes slides the wrong way). How many wins now? Try more episodes.
2. Print `Q.argmax(axis=1).reshape(4, 4)` to see the learned direction in each square (0 = left, 1 = down, 2 = right, 3 = up).
3. Set `epsilon = 0` from the start. Why does the agent fail to learn?

Part 5 — Engineering and MLOps

Chapter 21: Git and GitHub for ML projects

Git records every version of your project's files, so you can go back in time, try ideas safely on a side branch, and work with others without overwriting each other. **GitHub** (or GitLab, Bitbucket) hosts Git projects online. Every professional AI team uses them.

21.1 Core concepts

Concept	Meaning
Repository (repo)	A project folder whose history Git tracks
Commit	A saved snapshot of changes, with a message explaining why
Staging area	The changes you've chosen to include in the next commit
Branch	An independent line of work; main is the default
Merge	Combine one branch's changes into another
Remote	A copy of the repo on a server, usually called origin
Push / pull	Send your commits to the remote / fetch and merge others'
Clone	Download a full copy of a remote repo
Pull request (PR)	A request on GitHub to merge a branch, where teammates review the code

21.2 Setup

Install Git from git-scm.com (it's pre-installed on macOS and most Linux; Colab has it too). Then tell it who you are, once:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

21.3 The everyday workflow

```
mkdir titanic-model && cd titanic-model
git init                                # start tracking this folder

# ... create train.py, requirements.txt ...
git status                             # what changed?
git add train.py requirements.txt # stage the files
git commit -m "Add baseline random forest training script"
git log --oneline                     # see history
```

Connect to GitHub: create an empty repository on github.com, then:

```
git remote add origin https://github.com/<your-username>/titanic-model.git
git branch -M main
git push -u origin main
```

21.4 Branches and pull requests

Never experiment directly on `main`. Create a branch for each idea:

```
git switch -c try-gradient-boosting    # create and move to a new branch
# ... edit, run, evaluate ...
git add train.py
git commit -m "Try HistGradientBoosting: CV accuracy 0.83 vs 0.80"
git push -u origin try-gradient-boosting
```

On GitHub, open a **pull request** from that branch. A teammate reviews, automated tests run (21.7), and you merge. To update your local copy afterwards: `git switch main && git pull`.

If two people changed the same lines, Git reports a **merge conflict** and marks both versions in the file between <<<<<<, ===== and >>>>>>. Edit the file to keep the right version, delete the markers, then `git add` and `git commit`.

21.5 Useful undo commands

Situation	Command
Discard unsaved changes to a file	<code>git restore train.py</code>
Unstage a file	<code>git restore --staged train.py</code>
See what changed	<code>git diff (unstaged) / git diff --staged</code>
Undo a pushed commit safely	<code>git revert <commit-id></code> (adds a new commit that reverses it)
Look at an old version	<code>git switch --detach <commit-id></code> , then <code>git switch main</code> to come back

21.6 What not to commit

Git is for code and small text files. Keep these out with a `.gitignore` file in the repo's root:

```
# .gitignore
__pycache__/
.venv/
.ipynb_checkpoints/
data/          # large datasets
models/        # trained weights (*.pt, *.joblib)
.env           # API keys and passwords: NEVER commit secrets
wandb/
```

If you ever commit a secret (an API key), consider it leaked: revoke it and create a new one, even after deleting the file.

For data and models, use **DVC** (Data Version Control) or **Git LFS**, which store the big files elsewhere and keep a small pointer in Git; or a model registry such as MLflow or the Hugging Face Hub.

21.7 A typical ML repository

```
titanic-model/
├── README.md          # what it does, how to run it
├── requirements.txt    # exact library versions
├── .gitignore
└── notebooks/         # exploration
```

```

├── src/
│   ├── train.py      # trains and saves the model
│   ├── evaluate.py   # runs the evaluation set
│   └── serve.py       # the prediction API (Chapter 22)
├── tests/            # automated checks
├── Dockerfile         # Chapter 22
└── .github/workflows/ # CI: run tests on every pull request

```

Continuous integration (CI) with GitHub Actions runs your tests — and your model evaluations (Chapter 17) — automatically on every push, so a change that breaks something is caught before it's merged.

Write good commit messages: a short summary line in the imperative ("Add...", "Fix..."), and for experiments, the result ("CV accuracy 0.83").

Try it

1. Put your Chapter 8 code into a repo, commit it, and push it to GitHub.
2. Create a branch, change the model, open a pull request to yourself and merge it.
3. Deliberately create a merge conflict (edit the same line on two branches) and resolve it.

Chapter 22: Docker — packaging a model

"It works on my machine" is the classic deployment failure: a different Python version, a missing library.

Docker packages your code, its libraries and a minimal operating system into one **image** that runs identically on any computer or cloud.

22.1 Core concepts

Concept	Meaning	Analogy
Image	A read-only package: OS + libraries + your code + model	A recipe, or a class in Python
Container	A running instance of an image, isolated from the rest of the machine	A dish cooked from the recipe; an object
Dockerfile	A text file of instructions for building an image	The recipe's steps
Layer	Each instruction adds a cached layer; unchanged layers are reused	Pre-prepared ingredients
Registry	A place to store and share images (Docker Hub, GitHub, AWS ECR, Google Artifact Registry)	A recipe library
Port mapping	Connects a port inside the container to one on your	A door to the kitchen

Concept	Meaning	Analogy
	machine	
Volume	Storage that survives when a container is deleted	A pantry outside the kitchen

Containers are much lighter than virtual machines: they share the host's operating system kernel and start in seconds.

Install **Docker Desktop** from docker.com (Mac, Windows, Linux). Docker doesn't run inside Colab, so this chapter runs on your own computer.

22.2 Step 1: train and save the model

```
# train.py
import joblib, seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

raw = sns.load_dataset("titanic")[["survived", "pclass", "sex", "age", "sibsp",
                                   "fare", "embarked"]]
X, y = raw.drop(columns="survived"), raw["survived"]
prep = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median"), StandardScaler()),
    ["pclass", "age", "sibsp", "fare"]),
    ("cat", make_pipeline(SimpleImputer(strategy="most_frequent"),
    OneHotEncoder(handle_unknown="ignore")), ["sex", "embarked"]),
])
model = Pipeline([("prep", prep), ("model", RandomForestClassifier(n_estimators=300,
max_depth=5, random_state=42))])
model.fit(X, y) # final model: trained on all data after
evaluation
joblib.dump(model, "model.joblib")
```

22.3 Step 2: a prediction API with FastAPI

An **API** lets other programs send data and get predictions over HTTP. **FastAPI** is the standard Python choice.

```
# serve.py
import joblib
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Titanic survival API")
model = joblib.load("model.joblib") # load once, at start-up

class Passenger(BaseModel): # validates incoming data
    pclass: int
```

```

sex: str
age: float | None = None
sibsp: int = 0
fare: float
embarked: str = "S"

@app.get("/health")
def health():
    return {"status": "ok"}

@app.post("/predict")
def predict(p: Passenger):
    X = pd.DataFrame([p.model_dump()])
    return {"survival_probability": round(float(model.predict_proba(X)[0, 1]), 3)}

```

With requirements.txt pinning the versions:

```

fastapi==0.115.*
uvicorn==0.32.*
scikit-learn==1.5.*
pandas==2.2.*
joblib==1.4.*

```

Pin the scikit-learn version to the one you trained with (python -c "import sklearn; print(sklearn.__version__)"); a saved model may not load in a different version.

22.4 Step 3: the Dockerfile

```

# Dockerfile
FROM python:3.12-slim

WORKDIR /app

# Install libraries first: this layer is cached until requirements.txt changes
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Then copy the code and model
COPY serve.py model.joblib ./

EXPOSE 8000
CMD ["uvicorn", "serve:app", "--host", "0.0.0.0", "--port", "8000"]

```

22.5 Step 4: build, run, test

```

python train.py # creates model.joblib
docker build -t titanic-api:1.0 . # build the image
docker run -d -p 8000:8000 --name titanic titanic-api:1.0 # run it in the
background

```

```

curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"pclass": 1, "sex": "female", "age": 29, "fare": 100}'
# {"survival_probability": 0.97}

```

Open <http://localhost:8000/docs> in a browser: FastAPI generates an interactive page where you can try the API.

22.6 Everyday Docker commands

Command	Does
<code>docker ps / docker ps -a</code>	List running / all containers
<code>docker logs -f titanic</code>	Follow a container's output
<code>docker exec -it titanic bash</code>	Open a shell inside a running container
<code>docker stop titanic / docker rm titanic</code>	Stop / delete a container
<code>docker images / docker rmi <image></code>	List / delete images
<code>docker tag titanic-api:1.0 <user>/titanic-api:1.0 then docker push <user>/titanic-api:1.0</code>	Publish an image to a registry

22.7 Good practice

- **Small images:** use `-slim` base images; add a `.dockerignore` (like `.gitignore`) to keep data, `.git` and notebooks out.
- **Version tags:** tag images `1.0`, `1.1`... not just `latest`, so you can roll back.
- **No secrets in images:** pass API keys at run time, `docker run -e ANTHROPIC_API_KEY=...`, or through a secrets manager.
- **Big models:** rather than baking multi-gigabyte LLM weights into the image, download them at start-up or mount them as a volume (`-v /models:/models`).
- **GPUs:** base the image on an NVIDIA CUDA or PyTorch image and run with `docker run --gpus all ...`. For LLMs, ready-made serving images such as vLLM or Hugging Face TGI give fast, batched inference.
- **Several services** (API + database + vector store): describe them together in a `docker-compose.yml` and start them all with `docker compose up`.

Try it

1. Build and run the image, then call `/predict` for three different passengers.
2. Change `serve.py` to also return the predicted class, rebuild as `1.1`, and run both versions on different ports (`-p 8001:8000`).
3. Write a `.dockerignore` and compare image sizes with `docker images`.

Chapter 23: Kubernetes and MLOps

Docker runs one container on one machine. **Kubernetes** (often written **K8s**) runs many containers across many machines: it restarts crashed ones, spreads traffic between copies, adds copies when load rises, and rolls out new versions without downtime. **MLOps** is the wider practice of keeping models reliable once they're in production.


```

selector:
  matchLabels: {app: titanic-api}
template:
  metadata:
    labels: {app: titanic-api}
  spec:
    containers:
    - name: api
      image: titanic-api:1.0
      imagePullPolicy: IfNotPresent
      ports:
      - containerPort: 8000
      resources:
        requests: {cpu: "250m", memory: "256Mi"} # guaranteed share
        limits: {cpu: "1", memory: "512Mi"} # maximum allowed
      readinessProbe: # only send traffic when ready
        httpGet: {path: /health, port: 8000}
        initialDelaySeconds: 5
      livenessProbe: # restart if it stops responding
        httpGet: {path: /health, port: 8000}
        initialDelaySeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: titanic-api
spec:
  selector: {app: titanic-api}
  ports:
  - port: 80
    targetPort: 8000
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: titanic-api
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: titanic-api}
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 70} # add pods above 70% CPU

```

```

kubectl apply -f k8s.yaml # create everything
kubectl get pods # two pods, STATUS Running
kubectl port-forward service/titanic-api 8080:80
curl -X POST http://localhost:8080/predict -H "Content-Type: application/json" \
  -d '{"pclass": 3, "sex": "male", "fare": 8}'

kubectl delete pod <one-pod-name> # watch Kubernetes replace it
automatically

```

```
kubectl set image deployment/titanic-api api=titanic-api:1.1 # rolling update
kubectl rollout undo deployment/titanic-api # roll back if something breaks
kubectl logs -l app=titanic-api # logs from all pods
```

For GPU models, nodes with GPUs are added to the cluster and a pod requests one with resources: `{limits: {nvidia.com/gpu: 1}}`. Tools such as **KServe**, **Ray Serve** and **vLLM** add model-specific serving on top of Kubernetes.

23.4 The MLOps lifecycle

1. **Data**: versioned datasets and a documented pipeline for collecting and cleaning them.
2. **Experiment tracking**: log every run's code version, data version, hyperparameters and metrics (MLflow, Weights & Biases).
3. **Model registry**: store approved models with versions and stages (staging, production).
4. **CI/CD**: on every change, automatically run tests and the evaluation set; build the Docker image; deploy to staging, then production.
5. **Serving**: online (API, milliseconds), batch (score millions of rows overnight), or streaming (events as they arrive).
6. **Monitoring**: watch the system and the model (23.6).
7. **Retraining**: on a schedule or when monitoring shows decay (**continuous training**), then back to step 2.

23.5 Safe rollout strategies

Strategy	How it works	Why
Rolling update	Replace pods a few at a time	Kubernetes' default; no downtime
Canary	Send ~5% of traffic to the new model, compare, then increase	Limits damage from a bad model
Shadow	New model gets copies of real requests; its answers are logged, not used	Test on real data with zero user risk
A/B test	Split users between models and measure the business outcome	Proves the new model is actually better
Blue-green	Run old and new side by side, switch traffic at once	Instant rollback

23.6 Monitoring models in production

A model can be "up" and still be silently wrong. Monitor three levels:

- **System**: latency, error rate, throughput, CPU/GPU and memory use, cost.
- **Data: data drift** — incoming inputs no longer look like training data (new customer types, a changed sensor). Compare feature distributions with statistical tests or the population stability index (PSI).

- **Model: concept drift** — the relationship between inputs and outcome changes (fraud tactics evolve). Track accuracy once true outcomes arrive, and prediction distributions before they do.

For LLM applications (**LLMOps**), also version prompts like code, gate releases on your eval set (Chapter 17), trace each step of chains and agents, track token usage and cost, sample conversations for review, and apply guardrails (input/output filters) against prompt injection and unsafe content.

Try it

1. Deploy the API to minikube and call it through port-forward.
2. Scale manually with `kubectl scale deployment/titanic-api --replicas=4` and watch `kubectl get pods -w`.
3. Write down which drift you'd expect for a driver-risk model if a company switched from city delivery vans to long-haul trucks, and what you'd monitor.

Chapter 24: Responsible AI

A model that's accurate on average can still be unfair to some people, impossible to explain, leak private data, or be manipulated. Responsible AI is the set of practices that catch these problems before they cause harm. It's part of doing the job well, not an optional extra.

24.1 Where bias comes from

Source	Example
Historical bias	Past hiring data reflects past discrimination, and the model learns it
Representation bias	A face dataset with few darker-skinned faces performs worse on them
Measurement bias	Using "arrests" as a stand-in for "crime" when policing differs by neighbourhood
Proxy features	Postcode quietly standing in for ethnicity or income, even if those columns are removed
Feedback loops	A model's decisions shape the future data it's retrained on

24.2 Measuring fairness

Compute your metrics separately for each group (Chapter 9.7), and compare:

- **Demographic parity**: does each group receive positive predictions at a similar rate?
- **Equal opportunity**: among people who truly qualify, is the recall the same for each group?
- **Equalized odds**: are both the true-positive and false-positive rates similar across groups?
- **Calibration within groups**: does a 70% score mean 70% for everyone?

These definitions can conflict mathematically; you can't always satisfy all at once. Choosing which matters is a decision for people who understand the context, not only for engineers.

```
import pandas as pd
from sklearn.metrics import recall_score

# using best, X_test, y_test from Chapter 8
results = pd.DataFrame({"group": X_test["sex"], "y": y_test, "pred":
best.predict(X_test)})
for g, part in results.groupby("group"):
    print(g, "positive rate:", round(part["pred"].mean(), 2),
          "recall:", round(recall_score(part["y"], part["pred"]), 2))
```

The library **Fairlearn** automates these comparisons and offers mitigation methods.

24.3 Explainability

- **Global**: which features matter overall? Use feature importance (Chapter 6) or **permutation importance** — shuffle one column and see how much the score drops.
- **Local**: why *this* prediction? **SHAP** assigns each feature a contribution for each individual prediction; **LIME** fits a simple model around one example.
- **Inherently interpretable models**: for high-stakes decisions (credit, health, employment), a slightly less accurate but explainable model is often the right choice.
- **LLMs**: explaining large neural networks is an open research field (**mechanistic interpretability**). In practice, ask for cited sources (RAG) and log inputs and outputs for review.

```
from sklearn.inspection import permutation_importance

r = permutation_importance(best, X_test, y_test, n_repeats=10, random_state=42)
print(pd.Series(r.importances_mean,
index=X_test.columns).sort_values(ascending=False))
# pip install shap -> shap.Explainer for per-prediction explanations
```

24.4 Privacy

- Collect only the data you need, and remove or mask **personally identifiable information (PII)**: names, emails, IDs, precise locations.
- Models can **memorize** training data; LLMs have been shown to repeat rare training text. Don't train on secrets.
- **Differential privacy** adds carefully measured noise so no single person's data can be recovered from the model.
- **Federated learning** trains across devices or organizations without moving raw data to one place.
- Follow the law where you operate (e.g. GDPR in Europe, PIPEDA in Canada) and your organization's data policies.

24.5 Security

- **Adversarial examples**: tiny, invisible changes to an input that flip a model's prediction.
- **Data poisoning**: corrupting training data to plant a hidden behaviour.

- **Prompt injection:** text hidden in a web page, email or document telling an LLM to ignore its instructions. Treat all retrieved content as untrusted data, limit what tools an agent can use, and require human approval for sensitive actions.
- **Model and data theft:** protect API keys, rate-limit endpoints, log access.

24.6 Transparency and oversight

- **Model cards** document what a model is for, how it was trained and evaluated, its known limitations and groups it performs worse on. **Datasheets** do the same for datasets.
- **Human in the loop:** for consequential decisions, the model advises and a person decides, with a way for affected people to appeal.
- **Regulation** is growing: the EU AI Act, for example, sets requirements by risk level. Check the rules that apply to your country and industry.
- **Environmental cost:** large training runs use significant energy; prefer smaller models when they're good enough.

Try it

1. Run the fairness code above. Is the model's recall similar for men and women? What might explain a gap?
2. Write a one-page model card for your Titanic model: purpose, data, metrics, limitations.
3. Think of an AI system at your workplace. Who could it treat unfairly, and how would you test for it?

Appendices

Appendix A: Math essentials

You can build useful AI with very little math, but four areas help you understand what's happening: linear algebra (how data is represented), calculus (how models learn), probability and statistics (how uncertainty is handled), and a little information theory (the losses). Each idea below links to where it appears in the book.

A.1 Linear algebra: data as vectors and matrices

- A **vector** is a list of numbers: one passenger's features, or one token's embedding (Chapter 14).
- A **matrix** is a grid of numbers: a whole dataset (rows $\times$ features), or a layer's weights (Chapter 10).
- The **dot product** multiplies two vectors element by element and adds up: $a \cdot b = a_1b_1 + a_2b_2 + \dots$. It measures how aligned they are; it's the heart of every neuron and of attention ($Q \cdot K$, Chapter 15).
- **Matrix multiplication** applies many dot products at once. A neural network layer is one matrix multiply plus an activation.
- The **norm** (length) of a vector, and **cosine similarity** = dot product divided by both lengths (Chapter 14).

```

import numpy as np
a, b = np.array([1, 2, 3]), np.array([4, 5, 6])
print(a @ b) # dot product: 32
print(np.linalg.norm(a)) # length: 3.74
print(a @ b / (np.linalg.norm(a) * np.linalg.norm(b))) # cosine similarity: 0.97
W = np.random.randn(3, 2) # a layer: 3 inputs -> 2
outputs
print(a @ W) # the layer's output

```

A.2 Calculus: how learning works

- A **derivative** is the slope: how fast the output changes when you nudge the input.
- A **partial derivative** is the slope with respect to one input while the others stay fixed.
- The **gradient** is the vector of all partial derivatives: it points uphill. Gradient descent steps the opposite way (Chapter 3).
- The **chain rule** says the slope of a chain of functions is the product of their slopes. Backpropagation is the chain rule applied layer by layer (Chapter 10).

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

A.3 Probability

- A **probability distribution** assigns likelihoods to outcomes. An LLM's output is a distribution over the next token (Chapter 15).
- **Conditional probability** $P(A | B)$: the chance of A given that B happened.
- **Bayes' theorem** updates beliefs with evidence; it's behind naive Bayes (Chapter 5):

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

- **Expected value** is the probability-weighted average outcome; **variance** measures spread.
- The **normal (Gaussian) distribution** is the bell curve; noise in Chapters 3 and 19 is drawn from it.

A.4 Statistics

- **Mean vs median**: the median resists outliers (Chapter 4).
- **Standard deviation**: typical distance from the mean; used in standardization.
- **Correlation**: how two variables move together, from -1 to 1. Correlation is not causation.
- **Sampling and confidence intervals**: a test score from 178 examples is an estimate with uncertainty (Chapter 9.7).
- **Hypothesis testing and p-values**: is a difference (for example, in an A/B test) likely to be real or chance?

A.5 Information theory

- **Entropy**: how uncertain a distribution is. A fair coin has high entropy; a two-headed coin has zero. Used to choose tree splits (Chapter 6).
- **Cross-entropy**: how surprised your model's distribution is by the true outcome. It's the loss for classification and for LLMs (Chapters 5, 15, 16).

- **KL divergence:** how different two distributions are. RLHF uses it to keep the tuned model close to the original (Chapter 16).

$$H(p, q) = - \sum_x p(x) \log q(x)$$

A.6 Where to learn more

- [3Blue1Brown](#): visual video series on linear algebra, calculus and neural networks.
- [Khan Academy](#): free courses on every topic above, from the basics.
- *Mathematics for Machine Learning* by Deisenroth, Faisal and Ong: a free textbook written exactly for this purpose.

Appendix B: The concept checklist and where to go next

This is the full list of concepts an AI/ML student is expected to know, mapped to the chapter that teaches each one. Tick them off as you can explain them in your own words and write the code without looking.

B.1 Foundations (Chapters 1–3, Appendix A)

- ☐ AI vs ML vs deep learning vs generative AI
- ☐ Model, parameters, training, inference
- ☐ Python basics, NumPy, pandas, Matplotlib
- ☐ Loss functions (MSE, cross-entropy)
- ☐ Gradient descent and learning rate
- ☐ Vectors, matrices, dot products; derivatives, gradients, chain rule; probability, Bayes' theorem; basic statistics

B.2 Classic machine learning (Chapters 4–9)

- ☐ Features, labels, data types; EDA
- ☐ Missing values, outliers, encoding, scaling, feature engineering
- ☐ Train / validation / test split; data leakage
- ☐ Supervised, unsupervised, self-supervised, reinforcement learning
- ☐ Linear and logistic regression, Ridge, Lasso
- ☐ kNN, SVM, naive Bayes
- ☐ Decision trees, random forests, gradient boosting (XGBoost, LightGBM)
- ☐ k-means, hierarchical clustering, DBSCAN; PCA, t-SNE, UMAP; anomaly detection
- ☐ Overfitting, underfitting, bias–variance trade-off, regularization
- ☐ Cross-validation, hyperparameter tuning, pipelines
- ☐ Confusion matrix, precision, recall, F1, ROC-AUC, PR-AUC; MAE, RMSE, R²; baselines; thresholds; calibration

B.3 Deep learning (Chapters 10–13)

- ☐ Neurons, layers, activation functions

- ☐ Forward pass, backpropagation
- ☐ Epochs, batches, optimizers (SGD, Adam, AdamW), learning-rate schedules
- ☐ Dropout, weight decay, batch/layer norm, early stopping, data augmentation
- ☐ PyTorch tensors, autograd, the training loop, GPUs
- ☐ CNNs, pooling, transfer learning, fine-tuning
- ☐ RNNs, LSTMs, vanishing gradients, sequence-to-sequence

B.4 Generative AI and LLMs (Chapters 14–19)

- ☐ Tokenization (BPE), embeddings, semantic search, vector databases
- ☐ Self-attention (Q, K, V), multi-head attention, causal masking, positional encoding
- ☐ Transformer blocks, residual connections; encoder vs decoder models
- ☐ Sampling: temperature, top-k, top-p; scaling laws
- ☐ Pre-training, SFT, RLHF, DPO, reasoning via RL
- ☐ LoRA, QLoRA, when to fine-tune vs prompt vs RAG
- ☐ Perplexity, benchmarks, task eval sets, LLM-as-judge, RAG evaluation, red-teaming
- ☐ Prompt engineering, APIs, RAG pipelines, tool use, agents
- ☐ VAEs, GANs, diffusion models, CLIP, multimodal models

B.5 Reinforcement learning (Chapter 20)

- ☐ Agent, environment, state, action, reward, policy, value
- ☐ Exploration vs exploitation; Q-learning; DQN; policy gradients and PPO

B.6 Engineering, MLOps and responsibility (Chapters 21–24)

- ☐ Git: commits, branches, pull requests, merge conflicts, .gitignore
- ☐ Docker: images, containers, Dockerfiles, registries
- ☐ Serving a model as an API (FastAPI)
- ☐ Kubernetes: pods, deployments, services, autoscaling, rollouts
- ☐ Experiment tracking, model registry, CI/CD, monitoring, data and concept drift
- ☐ Fairness metrics, explainability (SHAP), privacy, security, prompt injection, model cards

B.7 Where to go next

Books, in the order suggested at the start of our conversation:

- *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, Aurélien Géron — more depth on Parts 2–3.
- *Understanding Deep Learning*, Simon Prince — free online; deeper theory for Parts 3–4.
- *Build a Large Language Model (From Scratch)*, Sebastian Raschka — extends Chapter 15 to a real GPT.
- *Hands-On Large Language Models*, Jay Alammar and Maarten Grootendorst — visual guide to using LLMs.
- *AI Engineering*, Chip Huyen — building and evaluating LLM products; pairs with Chapters 17, 18 and 23.

Courses and videos: Andrew Ng's *Machine Learning Specialization* and *Deep Learning Specialization*; fast.ai's *Practical Deep Learning for Coders*; Andrej Karpathy's *Neural Networks: Zero to Hero* (builds GPT step by step); the free Hugging Face courses on LLMs and diffusion.

Landmark papers to read once you're comfortable: *Attention Is All You Need* (2017, transformers), *BERT* (2018), *Language Models are Few-Shot Learners* (2020, GPT-3), *Training language models to follow instructions with human feedback* (2022, RLHF), *Denoising Diffusion Probabilistic Models* (2020), *LoRA* (2021).

Practice: Kaggle competitions and datasets; rebuild each chapter's project on data from your own work; publish your code on GitHub (Chapter 21). A portfolio of three finished projects — for example a tuned tabular model served in Docker, a RAG assistant with an evaluation set, and a small fine-tuned LLM — demonstrates most of this checklist.